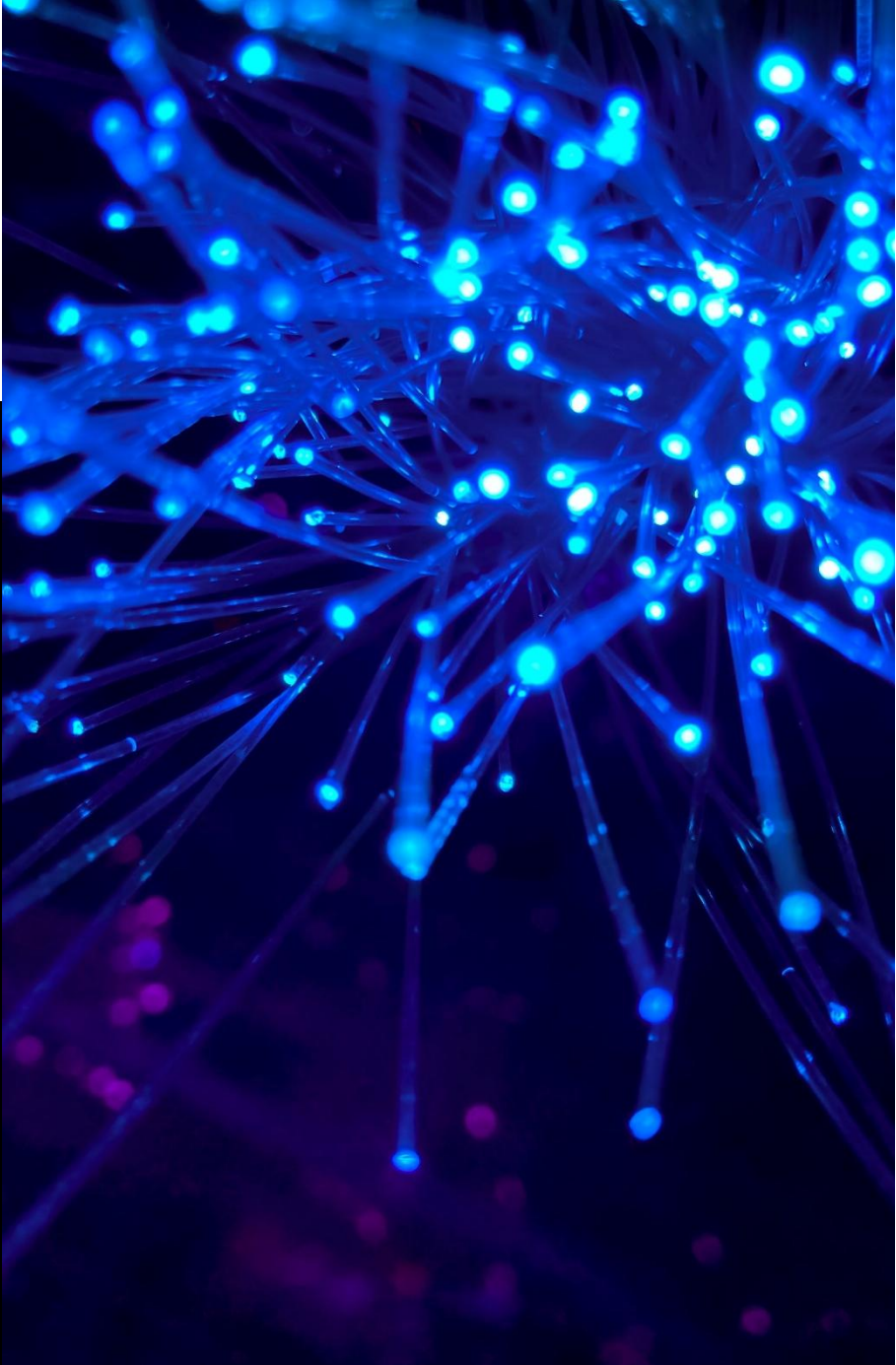




## Gestión de Datos

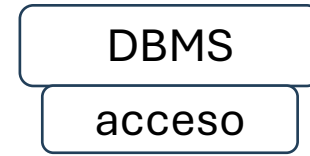
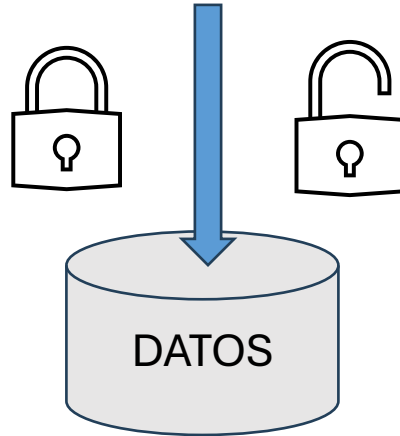
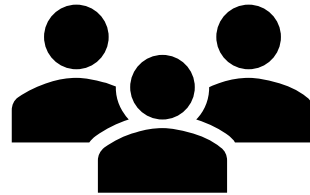
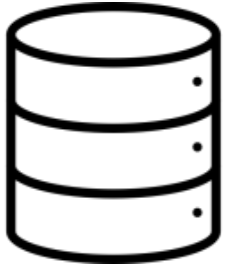
Fundamentos de Gestión de Datos



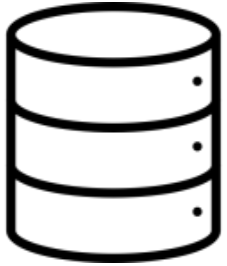
# Bases de datos

---

# Bases de datos



# Bases de datos



Recopilación organizada de datos estructurados o información.



Almacenada en un sistema informático.



Controlada por un sistema de gestión de base de datos (DBMS).

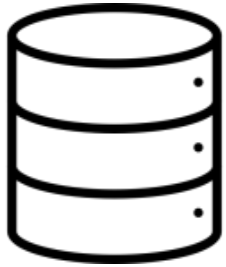


Permite la recuperación (rápida) de la información almacenada.



Análisis de datos.

# Bases de datos



Recopilación organizada de datos estructurados o información.



Almacenada en un sistema informático.



Controlada por un sistema de gestión de base de datos (DBMS).



Permite la recuperación (rápida) de la información almacenada.



Análisis de datos.

## Modelo de BD



<https://aws.amazon.com/es/what-is/database/>

Muestra la estructura lógica de una base de datos. Define las relaciones y las reglas que determinan el modo en que los datos se pueden almacenar, organizar y manipular.

# **Evolución BBDD**

# Evolución BBDD

- 0. Almacenado de registros
- 1. Almacenado secuencial de registros
- 2. BBDD jerárquicas
- 3. BBDD red
- 4. BBDD relacionales

- 5. BBDD orientadas a objetos
- 6. BBDD NoSQL
- 7. BBDD columnar
- 8. BBDD orientadas a grafos

9. BBDD de series temporales, BBDD de libro mayor y BBDD en memoria

10. BBDD autónomas



## 0. Almacenado de registros



[https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgoXP3yp3kK92pquryYp0QIvhlhr\\_pb2FOuV85stSCCMicuJD5\\_Hw9sOcWrd8P48Lk3RSxO5-KKgkGxMjBSYHtmDDslvWMbdSdAGytTq4VzEtCWleksiC6DJUVnDoNQ\\_ttPMO3jzZX9M/s200/varas.Leon.jpg](https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgoXP3yp3kK92pquryYp0QIvhlhr_pb2FOuV85stSCCMicuJD5_Hw9sOcWrd8P48Lk3RSxO5-KKgkGxMjBSYHtmDDslvWMbdSdAGytTq4VzEtCWleksiC6DJUVnDoNQ_ttPMO3jzZX9M/s200/varas.Leon.jpg)



[https://commons.wikimedia.org/wiki/File:Piedra\\_del\\_Sol.png](https://commons.wikimedia.org/wiki/File:Piedra_del_Sol.png)



<https://media.serverando.de/media/8f/a8/d5/1721651685/11.webp?ts=1721834468>

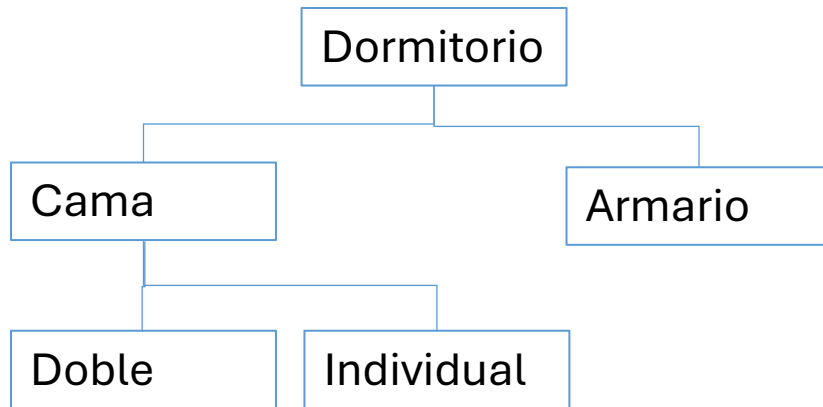


## 1. Almacenado secuencial de registros

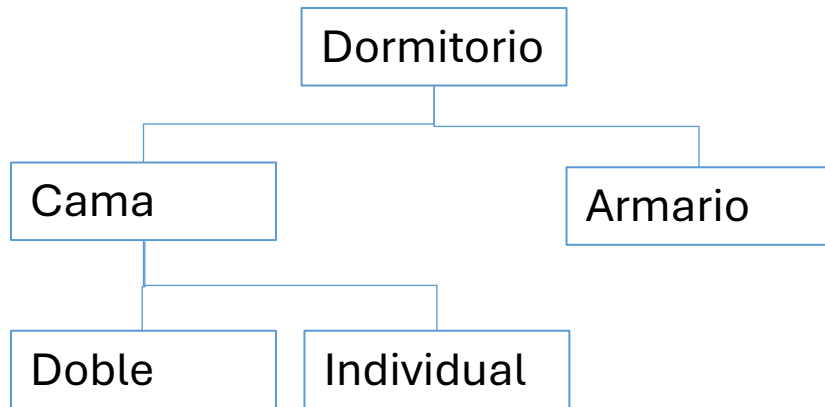


|    |              |                 |                     |                        |                        |            |              |  |
|----|--------------|-----------------|---------------------|------------------------|------------------------|------------|--------------|--|
| 4  |              |                 |                     |                        |                        |            |              |  |
| 5  | <b>Fecha</b> | <b>Concepto</b> | <b>Total gastos</b> | <b>Importe SIN IVA</b> | <b>Importe CON IVA</b> | <b>Iva</b> | <b>Total</b> |  |
| 6  |              |                 |                     |                        |                        |            |              |  |
| 7  | Enero        | Gastos          | 0                   | 0                      | 0                      | 0          | 0            |  |
| 8  | Febrero      | Gastos          | 20,686 × 0          | 0                      | 0                      | 0          | 0            |  |
| 9  | Marzo        | Gastos          | =+D9+E9             | 16,780                 | 3,906                  | 625        | 4,530        |  |
| 10 | Abril        | Gastos          | 13,616              | 13,530.00              | 86                     | 14         | 100          |  |
| 11 | Mayo         | Gastos          | 5,701               | 5,475.00               | 226                    | 36         | 262          |  |
| 12 | Junio        | Gastos          | 1,295               | 0                      | 1,295                  | 207        | 1,503        |  |
| 13 | Julio        | Gastos          | 0                   | 0                      | 0                      | 0          | 0            |  |
| 14 | Agosto       | Gastos          | 2,361               | 1,905                  | 456                    | 73         | 530          |  |
| 15 | Septiembre   | Gastos          | 9,309               | 5,324                  | 3,985                  | 638        | 4,623        |  |
| 16 | Octubre      | Gastos          | 543                 | 0                      | \$543.22               | 87         | 630          |  |
| 17 | Noviembre    | Gastos          | 7,862               | 0                      | 7,862                  | 1,258      | 9,120        |  |
| 18 | Diciembre    | Gastos          | 0                   |                        |                        | 0          | 0            |  |
| 19 |              |                 | 61,374              | 43,014                 | 18,360                 | 2,938      | 21,297       |  |
| 20 |              |                 |                     |                        |                        |            |              |  |

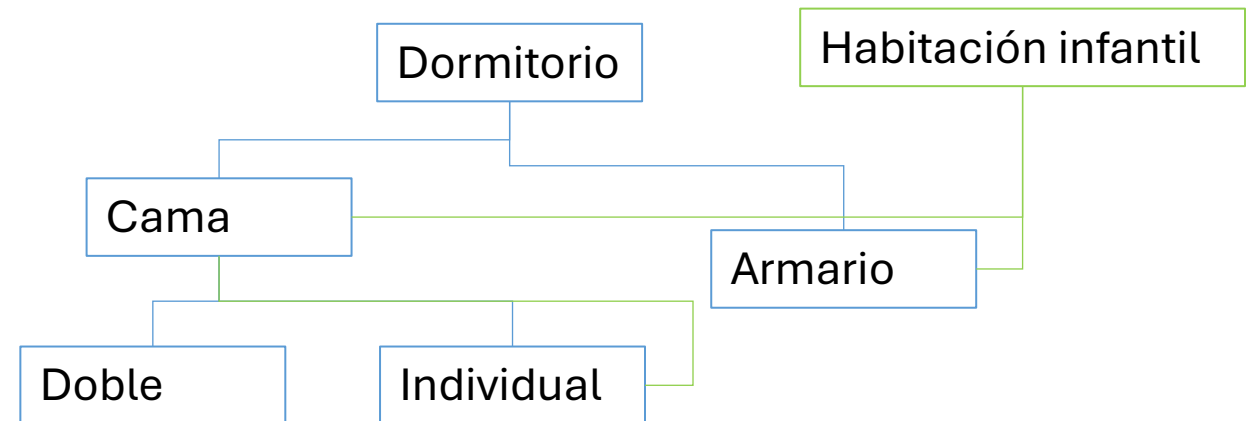
## 2. BD jerárquica: estructura de árbol

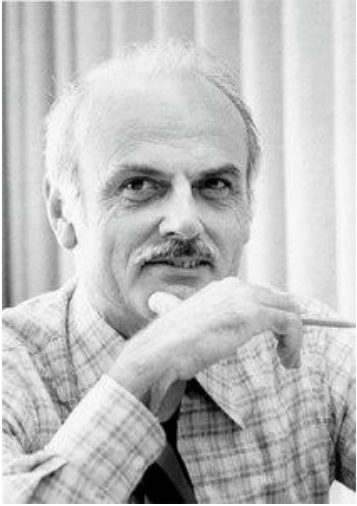


## 2. BD jerárquica: estructura de árbol



## 3. BD red: estructura de red





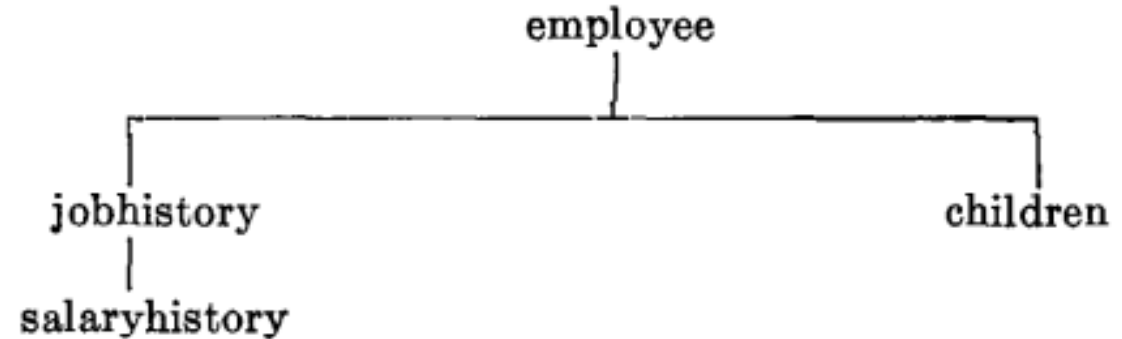
Edgar Frank Codd  
1923 England

## Information Retrieval

### A Relational Model of Data for Large Shared Data Banks

E. F. Codd  
IBM Research Laboratory, San Jose, California

"A Relational Model of Data for  
Large Shared Data Banks" **1970**



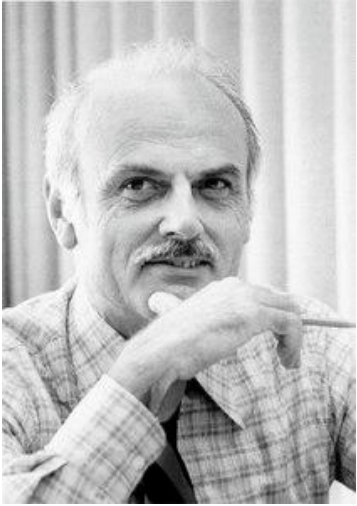
employee (*man#*, name, birthdate, jobhistory, children)  
 jobhistory (*jobdate*, title, salaryhistory)  
 salaryhistory (*salarydate*, salary)  
 children (*childname*, birthyear)

FIG. 3(a). Unnormalized set

employee' (*man#*, name, birthdate)  
 jobhistory' (*man#*, *jobdate*, title)  
 salaryhistory' (*man#*, *jobdate*, *salarydate*, salary)  
 children' (*man#*, *childname*, birthyear)

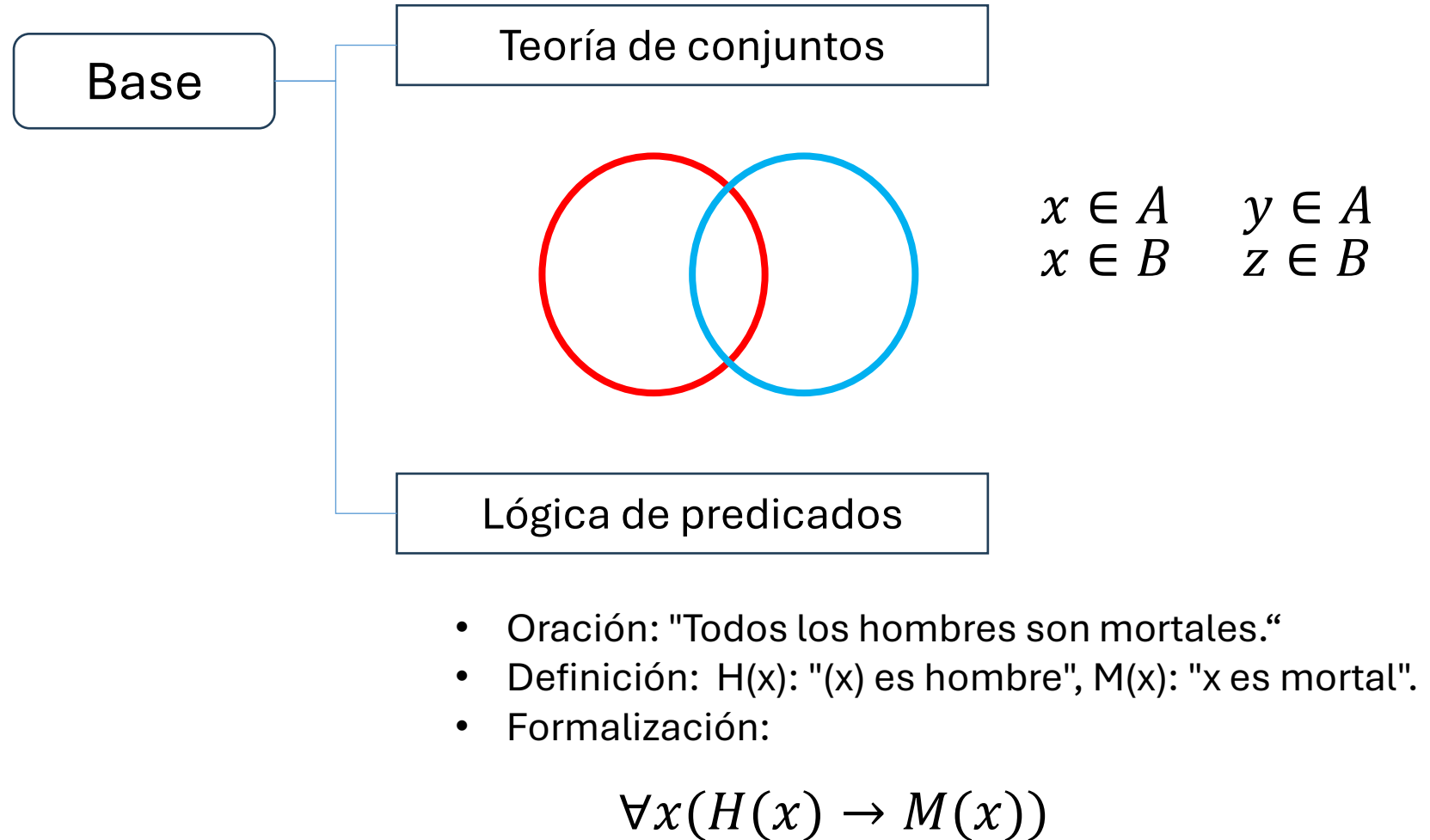
FIG. 3(b). Normalized set

# Evolución BBDD



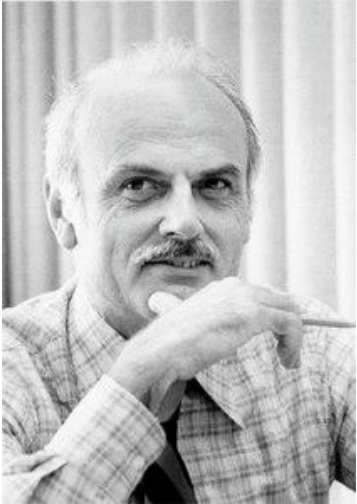
Edgar Frank Codd  
1923 England

*"A Relational Model  
of Data for Large  
Shared Data Banks"*  
**1970**





# Evolución BBDD



Edgar Frank Codd  
1923 England

*"A Relational Model  
of Data for Large  
Shared Data Banks"*  
**1970**

## Logro

Revolucionó la gestión de datos al organizarlos en tablas (relaciones) de filas y columnas, sustituyendo estructuras jerárquicas rígidas.

Propuso separar la vista lógica de los datos de su almacenamiento físico, permitiendo cambios sin afectar a las aplicaciones.

cada relación es un conjunto de datos, el orden en el que estos se almacenen no tiene relevancia ( $\neq$  modelos jerárquico y de red)

## Componentes:

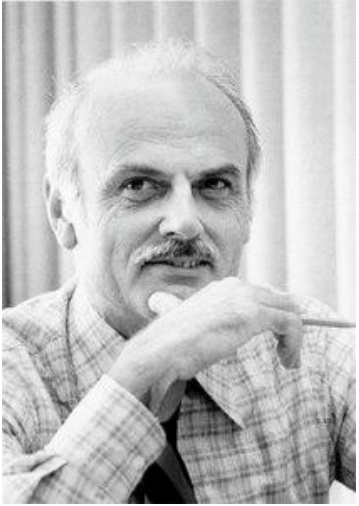
- Relaciones: tablas de datos
- Tupla: fila o registro
- Atributo: columna o campo

## Manipulación: precursor SQL

- Operaciones relacionales para consultar y actualizar datos

Gestión más flexible de grandes bancos de datos

# Evolución BBDD



Edgar Frank Codd  
1923 England

*"A Relational Model  
of Data for Large  
Shared Data Banks"*  
**1970**



Los datos se almacenan  
como relaciones



Colección de relaciones

- ✓ Sencilla
- ✓ Integridad de datos
- ✓ Múltiples usuarios a la vez
- ✓ Nulos
- ✓ CRUD

# IBM

International Business  
Machines Corporation

Bases de datos jerárquicas  
creadas para la NASA: IBM  
Information Management  
System (IMS)

## Las 12 reglas de Codd

- **0: Fundamento de la base de datos.** Un sistema relacional debe ser capaz de gestionar sus bases de datos enteramente a través de sus capacidades relacionales.
- **1: Regla de la información.** Toda la información en la base de datos se representa de una sola manera: valores en tablas (filas y columnas).
- **2: Regla de acceso garantizado.** Todos los datos deben ser accesibles lógicamente a través de una combinación de nombre de tabla, valor de clave primaria y nombre de columna.
- **3: Tratamiento sistemático de valores nulos.** El sistema debe ser capaz de manejar valores nulos (diferentes de cero, cadena vacía o espacios) de manera sistemática para representar información faltante o inaplicable.
- **4: Catálogo dinámico en línea.** La descripción de la base de datos se almacena a nivel lógico igual que los datos, permitiendo a usuarios autorizados consultar la estructura (metadatos) con el mismo lenguaje.
- **5: Regla del sublenguaje de datos completo.** El sistema debe soportar al menos un lenguaje relacional que permita definir, manipular y gestionar la seguridad e integridad de los datos, generalmente SQL.
- **6: Regla de actualización de vistas.** Todas las vistas que son teóricamente actualizables deben poder ser actualizadas por el sistema.
- **7: Inserción, actualización y borrado de alto nivel.** El sistema debe permitir manipular conjuntos de datos (filas) y no solo registros individuales.
- **8: Independencia física de datos.** Los programas de aplicación no deben verse afectados si se producen cambios en las estructuras físicas de almacenamiento.
- **9: Independencia lógica de datos.** Los programas de aplicación no deben verse afectados por cambios en las tablas lógicas.
- **10: Independencia de integridad.** Las restricciones de integridad deben almacenarse en el catálogo, no en los programas de aplicación.
- **11: Independencia de distribución.** La distribución de partes de la base de datos en diferentes sitios no debe afectar a los usuarios o aplicaciones.
- **12: Regla de no subversión.** Si el sistema tiene una interfaz de bajo nivel (registro a registro), no debe utilizarse para romper o alterar las reglas relacionales.

# Evolución BBDD



Ed Oates, Bob Miner y Larry Ellison  
1977 SDL (Software Development Laboratories)



1979-1983



1983-1995



1995-PRESENT

1979 Primera base de datos relacional

Clientes: Fuerza Aérea y CIA  
nombre en clave: Oracle

1982: SDL -> Oracle Systems Corporation

## 4. Bases de datos relacionales (80s)

Los datos se organizan en tablas con una serie de filas y columnas.

- Las filas o tuplas representan registros individuales, por ejemplo, un registro temporal, un producto, un usuario, ...
- Las columnas representan atributos del registro o entidad reflejada en la fila.

Cientes

| ID_cliente | Nombre_cli   | Dir_facturación    | Dir_envio          | ... |
|------------|--------------|--------------------|--------------------|-----|
| 8458       | Pepe Pérez   | C/ Arapiles 13 ... | C/ Arapiles 13 ... |     |
| 8459       | María Martín | C/ Arapiles 13 ... | C/ Arapiles 13 ... |     |

Pedidos

| ID_pedido | ID_cliente | Estado_pedido  | Fec_pedido          | ... |
|-----------|------------|----------------|---------------------|-----|
| 834581    | 8458       | En preparación | 2025-03-01 14:31:03 |     |
| 834592    | 8459       | Entregado      | 2025-03-02 14:31:03 |     |

Los datos se pueden relacionar entre sí. Lo hacen a través de claves:

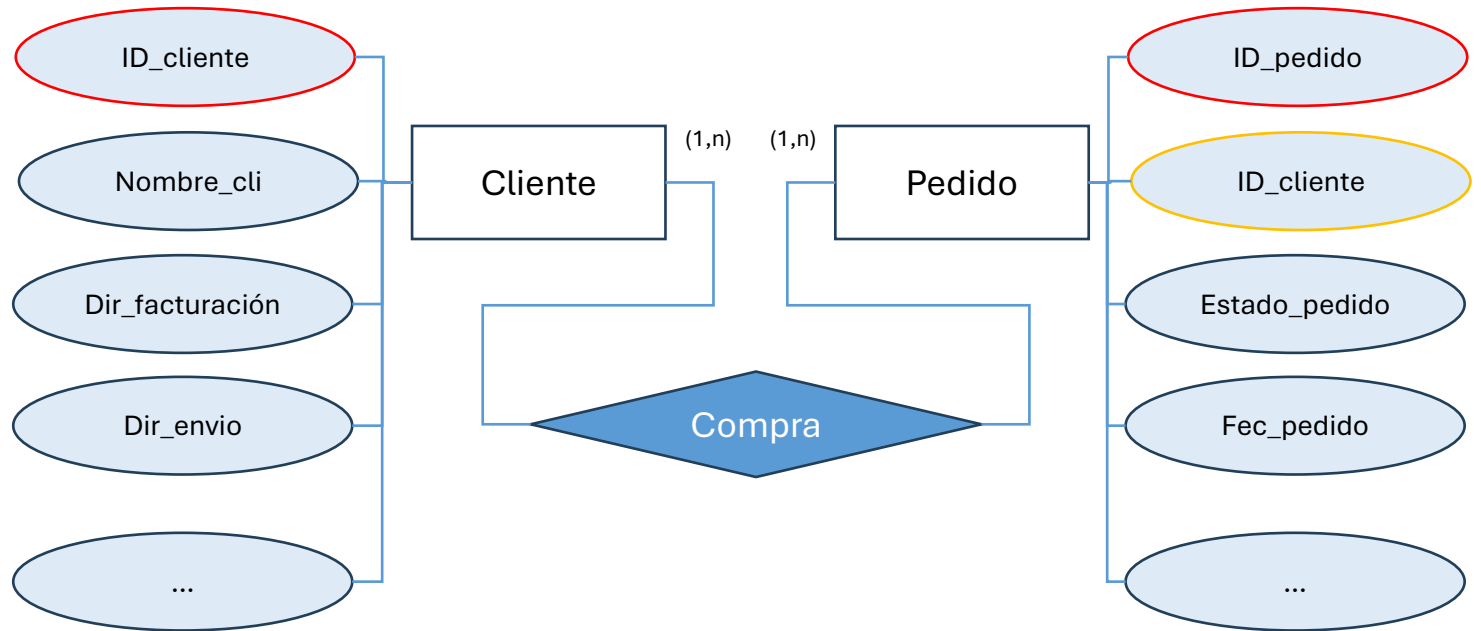
- Primarias: identifican de manera única las filas.
- Foráneas: relacionan la tabla con la clave primaria de otra tabla.

## 4. Bases de datos relacionales (80s)

### Modelo entidad-relación

Diagrama entidad-relación:  
representación gráfica

- Entidades: conceptos de los que trata la bbdd.
- Atributos: describen las propiedades.
- Relaciones: vínculos entre parejas de entidades.





## 4. Bases de datos relacionales (80s)

Los datos se organizan como relaciones predefinidas, por lo que pueden consultarse de forma declarativa.

Una consulta declarativa es una forma de definir lo que deseas extraer del sistema sin expresar cómo debería calcularse el resultado.

El lenguaje de consulta estructurado (SQL) es un lenguaje de programación diseñado específicamente para gestionar y administrar bases de datos relacionales.

Es por lo tanto un lenguaje específico de dominio (DSL, Domain Specific Language).

Lenguaje de

- definición de datos
- manipulación de datos
- control de datos

# Evolución BBDD

- 0. Almacenado de registros
- 1. Almacenado secuencial de registros
- 2. BBDD jerárquicas
- 3. BBDD red
- 4. BBDD relacionales

- 5. BBDD orientadas a objetos
- 6. BBDD NoSQL
- 7. BBDD columnar
- 8. BBDD orientadas a grafos

- 9. BBDD de series temporales, BBDD de libro mayor y BBDD en memoria

- 10. BBDD autónomas

## 5. Bases de datos orientadas a objetos (90s)

Asignación de atributos a un objeto, derivado de la programación orientada a objetos

Son sistemas más flexibles, más allá de las estructuras tabulares típicas de SQL.

Con un enfoque dirigido al almacenamiento y la gestión de datos.

Mesa

- Material
- Color
- Tamaño

Ejemplos: Oracle y Microsoft SQL Server

## \*\*Manifiesto Ágil

"Estamos descubriendo mejores formas de desarrollar software al hacerlo y ayudar a que otros lo hagan.

A través de este trabajo hemos llegado a valorar:

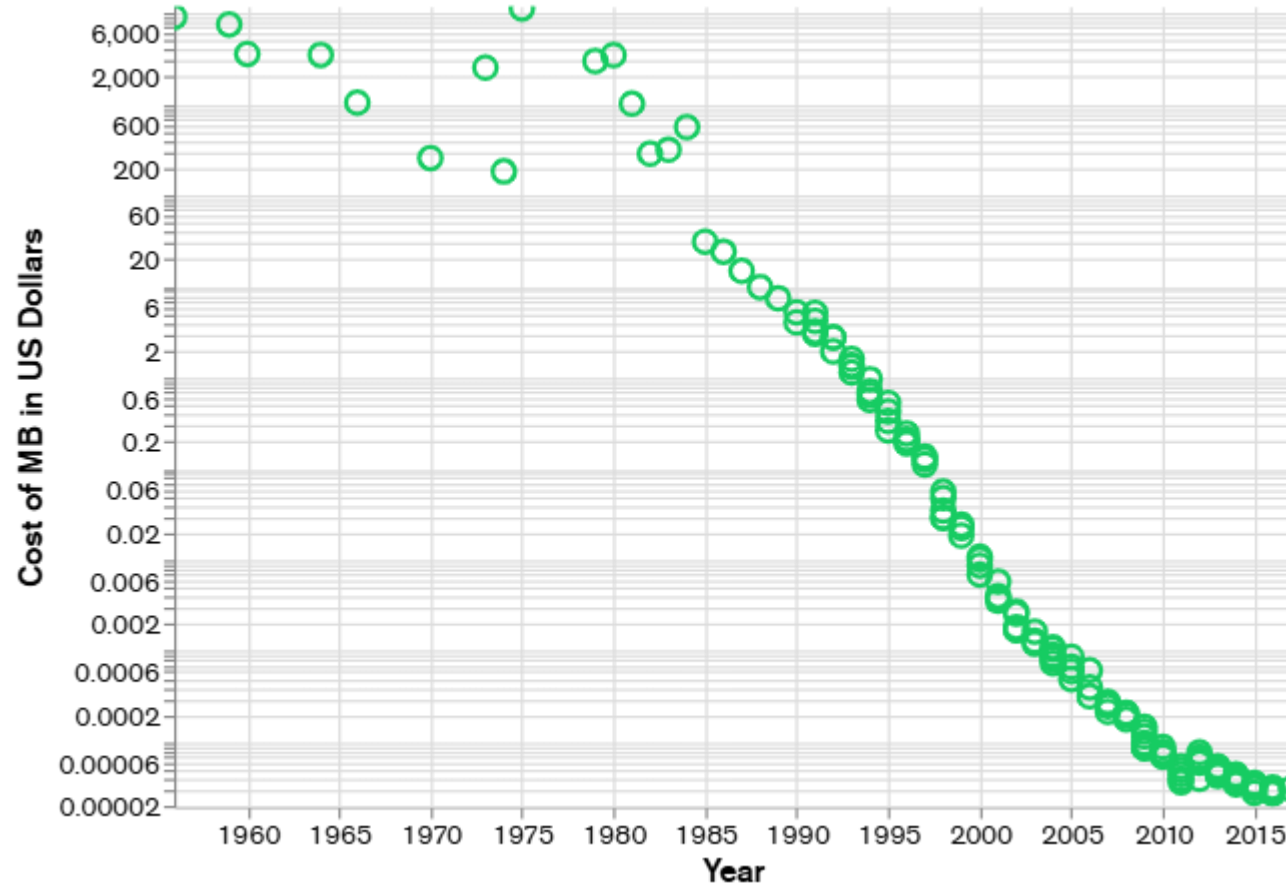
"las personas e interacciones sobre los procesos y las herramientas;  
el software funcional sobre la documentación completa  
; colaboración con los clientes sobre la negociación del contrato; y  
responder al cambio sobre el seguimiento de un plan".

"Es decir, si bien hay valor en los elementos  
de la derecha, valoramos más los elementos de la izquierda".

Valores del Manifiesto Agile:

1. Individuos e interacciones sobre procesos y herramientas
2. Software de trabajo sobre la documentación completa
3. Colaboración del cliente sobre la negociación del contrato
4. Responder al cambio por encima de seguir un plan

# Evolución BBDD



<https://www.mongodb.com/es/resources/basics/databases/nosql-explained>

Las bases de datos NoSQL surgieron a finales de la década de 2000 a medida que el costo del almacenamiento disminuía significativamente.

## 6. Bases de datos NoSQL (sXXI): no sólo SQL

### Origen

- 1998 Carlo Strozzi utiliza el término por primera vez para su base de datos, una open-source ligera que no ofrecía una interfaz SQL, pero sí seguía el sistema relacional.
- 2009 Eric Evans reintroduce el término en el entorno de las discusiones sobre BBDD distribuidas de código abierto que se salen del esquema del modelo relacional.
- Crecen con las principales redes sociales (Facebook, Twitter, Google y Amazon): al crecer la web en tiempo real se crea una necesidad de proporcionar información procesada a partir de grandes volúmenes de datos con unas estructuras horizontales similares => importa más el rendimiento (tiempo real) que la coherencia (que requiere una gran cantidad del tiempo del proceso).

## 6. Bases de datos NoSQL (sXXI): no sólo SQL

### Origen

- 1998 no estándar SQL
- 2005 primeros pasos de Google Bigtable: de columna ancha, disponible a través de Google Cloud en 2015.
- 2009 primera BD de documentos, MongoDB
- 2010 aparecen más BD NoSQL: Apache Hbase (columna ancha), Cassandra (columna ancha), Couchbase (clave-valor), Neo4J (grafos)
- 2012 Amazon DynamoDB (clave-valor)
- 2016 Azure CosmosDB (clave-valor) -> extendida para distintos tipos



## 6. Bases de datos NoSQL (sXXI): no sólo SQL

Bases de datos **no relacionales** que permiten almacenar datos semiestructurados o no estructurados.

- No requieren esquemas de tablas fijos
- Posibilidad de escalar horizontalmente -> fragmentación en servidores
- Posibilidad de almacenar datos duplicados



<https://www.mongodb.com/es/compan-y/what-is-mongodb>



<https://redis.io/>

Datos clave:valor  
Datos documentos tipo JSON  
(JavaScript Object Notation)

## 7. Bases de datos columnares

- Arquitectura distribuida y redundante: los datos se replican en varios nodos
- Los datos se almacenan en forma clave:valor en columnas (modelo híbrido). Se puede definir como tablas con columnas y filas, pero no es como se almacena.

|                 |                    |                       |                               |
|-----------------|--------------------|-----------------------|-------------------------------|
| ID de cliente 1 | Columna: Nombre    | Columna: País         |                               |
|                 | Valor: Juan García | Valor: Estados Unidos |                               |
| ID de cliente 2 | Columna: Nombre    | Columna: Edad         | Columna: Correo electrónico   |
|                 | Valor: María Pérez | Valor: 35             | Valor: mariaperez@ejemplo.com |

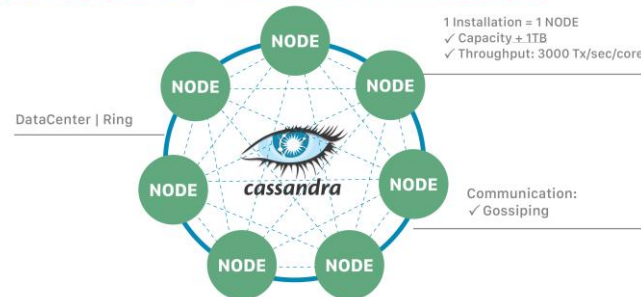
Ejemplo de estructura de una tabla de columna ancha

## 7. Bases de datos columnares

- Arquitectura distribuida y redundante: los datos se replican en varios nodos
- Los datos se almacenan en forma clave:valor en columnas (modelo híbrido). Se puede definir como tablas con columnas y filas, pero no es como se almacena.



ApacheCassandra™ = NoSQL Distributed Database

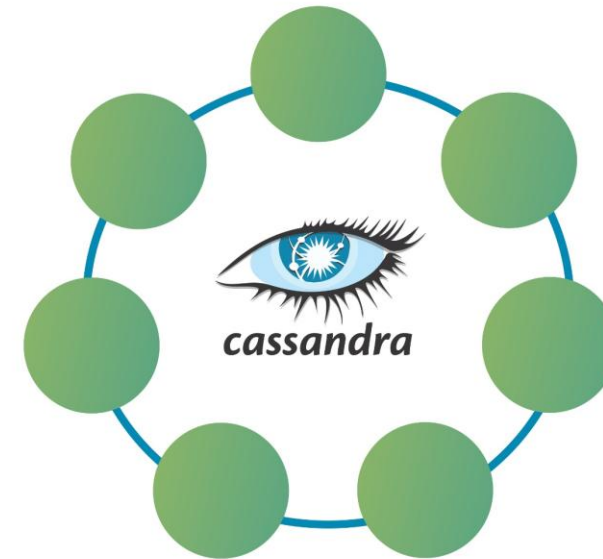
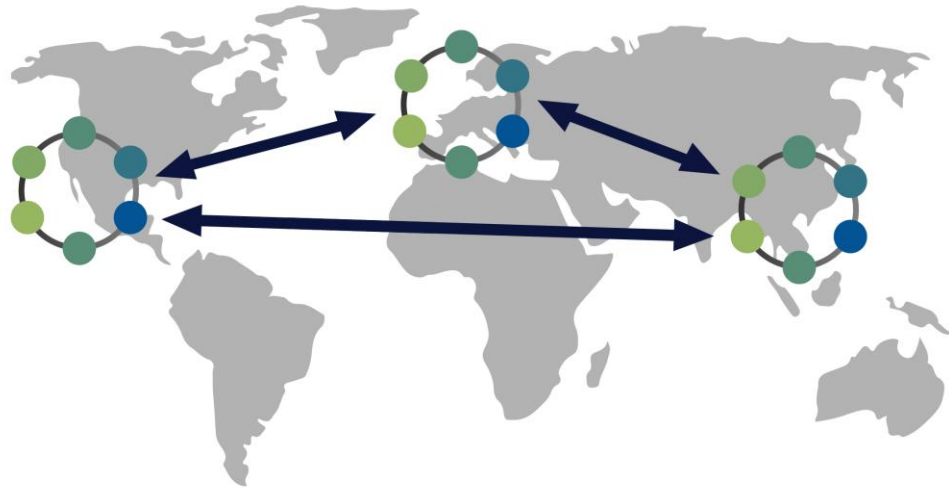


[https://cassandra.apache.org/\\_/cassandra-basics.html](https://cassandra.apache.org/_/cassandra-basics.html)

- BD distribuida
- Lenguaje propio denominado CQL (Cassandra Query Language): similar a SQL
- No soporta join o subqueries
- Particiona las filas de acuerdo a la clave primaria de partición, reorganizando los datos en tablas agrupadas por columnas (familias de columnas ↔ tablas)

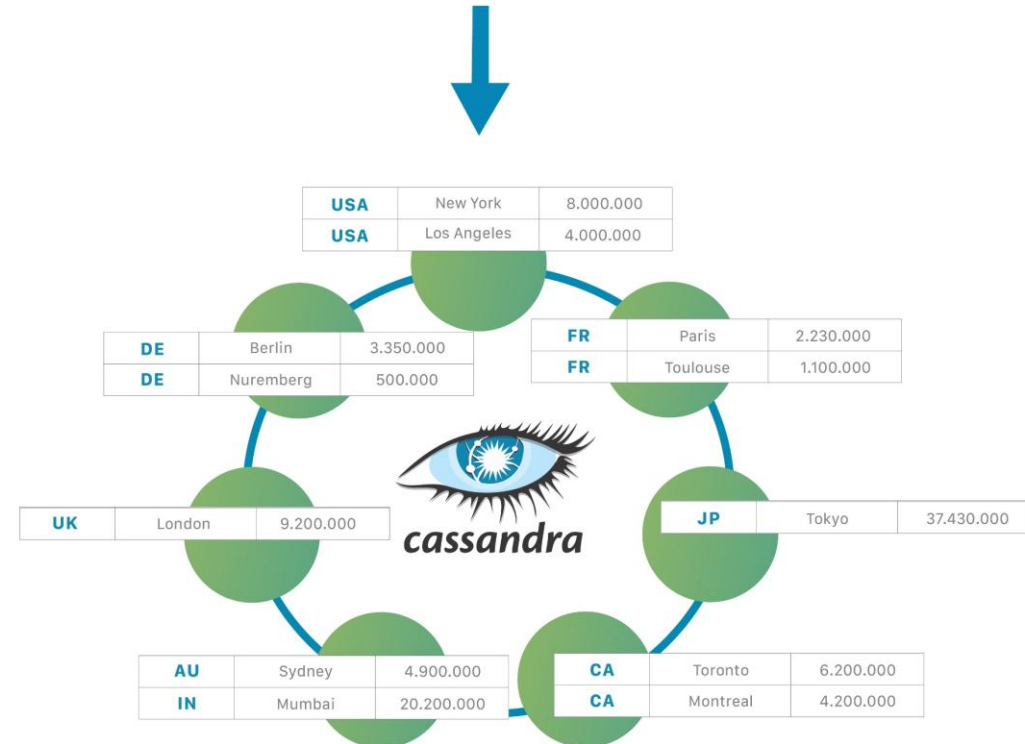
```
Create table MusicPlaylist
(
    SongId int,
    SongName text,
    Year int,
    Singer text,
    Primary key((SongId, Year), SongName)
);
```

## 7. Bases de datos columnares



| COUNTRY | CITY        | POPULATION |
|---------|-------------|------------|
| USA     | New York    | 8.000.000  |
| USA     | Los Angeles | 4.000.000  |
| FR      | Paris       | 2.230.000  |
| DE      | Berlin      | 3.350.000  |
| UK      | London      | 9.200.000  |
| AU      | Sydney      | 4.900.000  |
| DE      | Nuremberg   | 500.000    |
| CA      | Toronto     | 6.200.000  |
| CA      | Montreal    | 4.200.000  |
| FR      | Toulouse    | 1.100.000  |
| JP      | Tokyo       | 37.430.000 |
| IN      | Mumbai      | 20.200.000 |

Partition Key

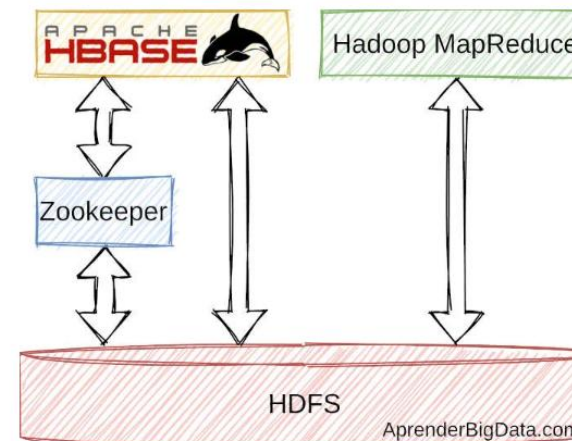


## 7. Bases de datos columnares

- Arquitectura distribuida y redundante: los datos se replican en varios nodos
- Los datos se almacenan en forma clave:valor en columnas (modelo híbrido)



<https://hbase.apache.org/book.html>



Esquema de Apache HBase

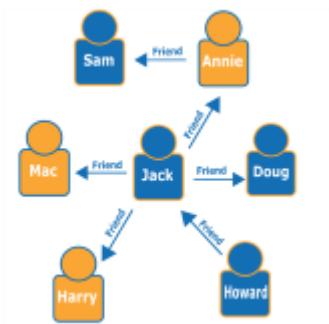
The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models.

## 8. Bases de datos orientadas a grafos (NoSQL)

Almacena y explora relaciones entre entidades.

Componentes: un nodo puede tener n relaciones

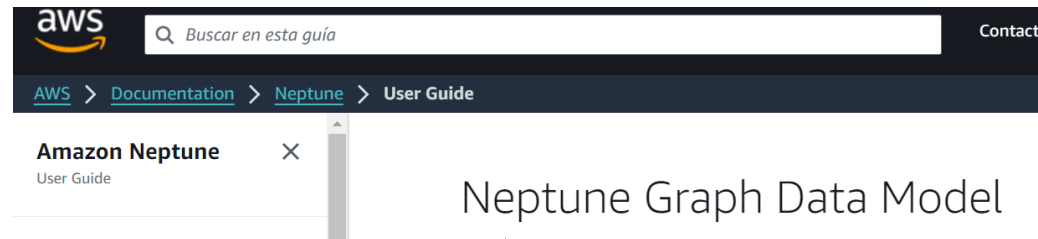
- Nodos: objetos de datos
- Bordes: relaciones entre objetos



Ejemplo de grafo de red social

<https://aws.amazon.com/es/nosql/graph/>

- Valor de los nodos
  - Rutas entre nodos: más larga / más corta / ...
  - Número de saltos entre nodos
- \* PATRONES



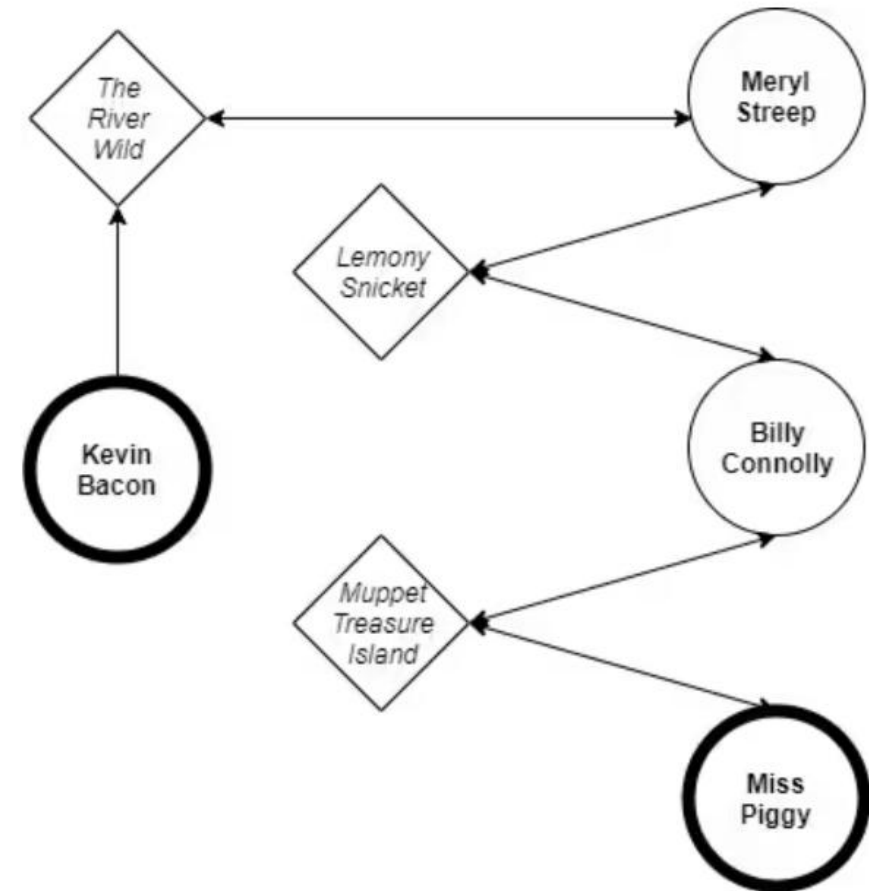
<https://docs.aws.amazon.com/neptune/latest/userguide/feature-overview-data-model.html>

## 8. Bases de datos orientadas a grafos (NoSQL)

<https://www.oracle.com/mx/autonomous-database/what-is-graph-database/>

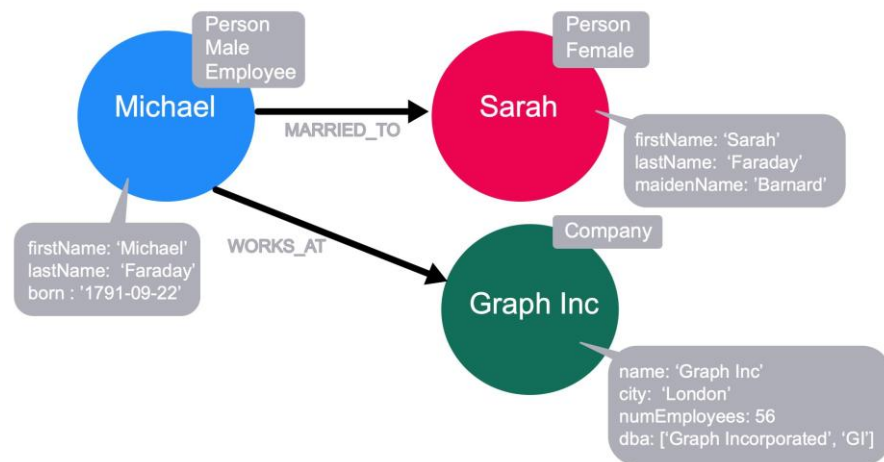
Grafos RDF (Resource Description Framework)

- una sentencia está representada por tres elementos: dos vértices conectados por un borde que representa el sujeto, el predicado y el objeto de una oración
- cada vértice y borde se identifica con un URI exclusivo (identificador único de recurso)





## 8. Bases de datos orientadas a grafos (NoSQL)



<https://neo4j.com/>

- Base de datos orientada a grafos nativa: implementa un modelo de grafos real desde el nivel de almacenamiento -> almacena nodos y relaciones en lugar de tablas o documentos.
- Lenguaje de consulta: Cypher®, lenguaje de consulta declarativo con cierta similitud con SQL.

## Resumiendo: Bases de datos NoSQL

### Tipos principales



Key-value  
database

#### Almacenes Clave-Valor

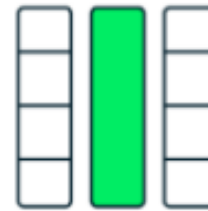
Amazon DynamoDB,  
Apache CouchDB



Document  
database

#### Basadas en Documentos

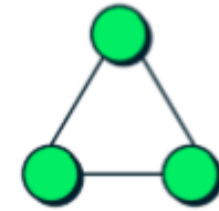
MongoDB



Wide-column  
database

#### Columna ancha

Google BigTable,  
Apache Cassandra

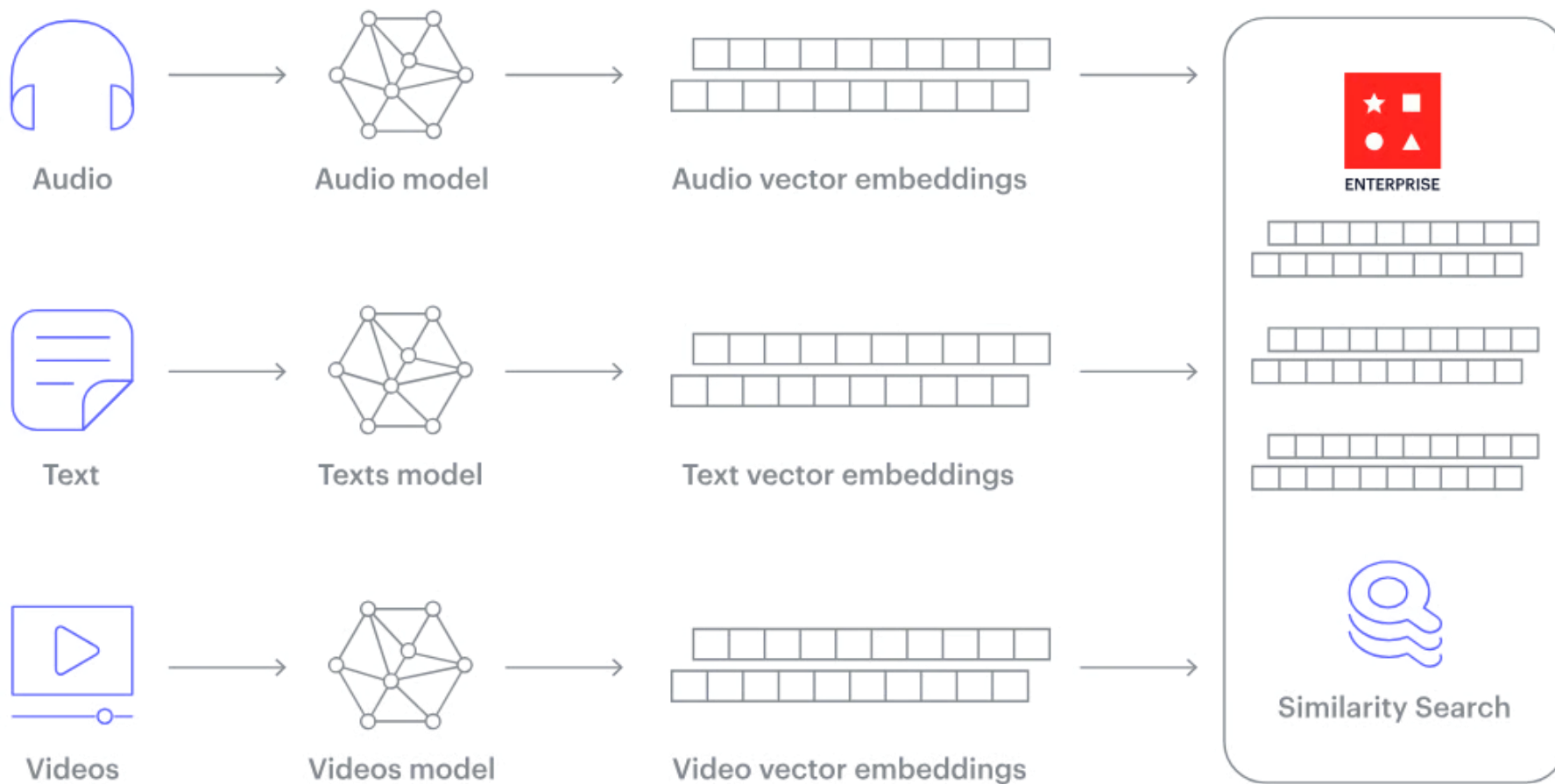


Graph database

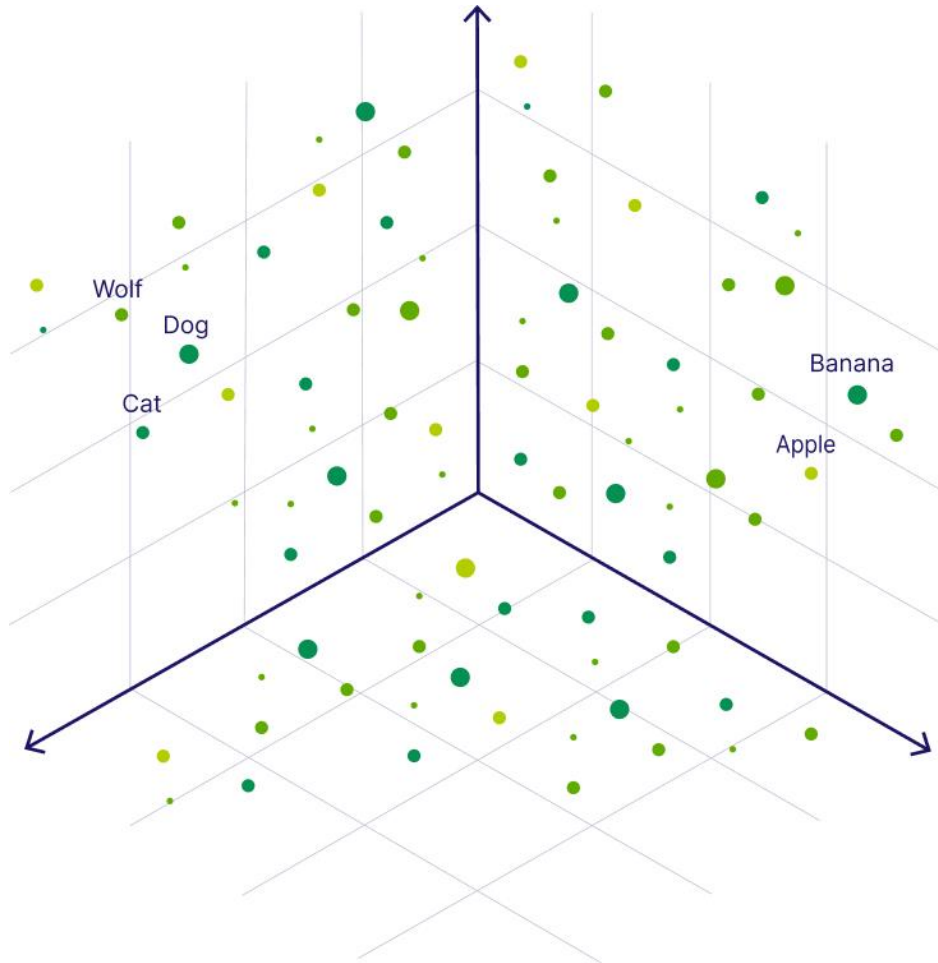
#### Grafos

Neo4J

## Bases de datos vectoriales



## Bases de datos vectoriales



Un vector (en el contexto de la inteligencia artificial) es simplemente una lista de números que representa el significado semántico de una imagen, una frase, audio e incluso vídeo.

Esta secuencia de números ayuda a los ordenadores a comprender en qué se parece cada dato: colocan los datos (como palabras o imágenes) como puntos en un "mapa matemático" en el que los conceptos similares están más cerca unos de otros.

Ej. como coordenadas GPS.

## Bases de datos vectoriales

BBDD vectoriales \_ Búsquedas de similitud

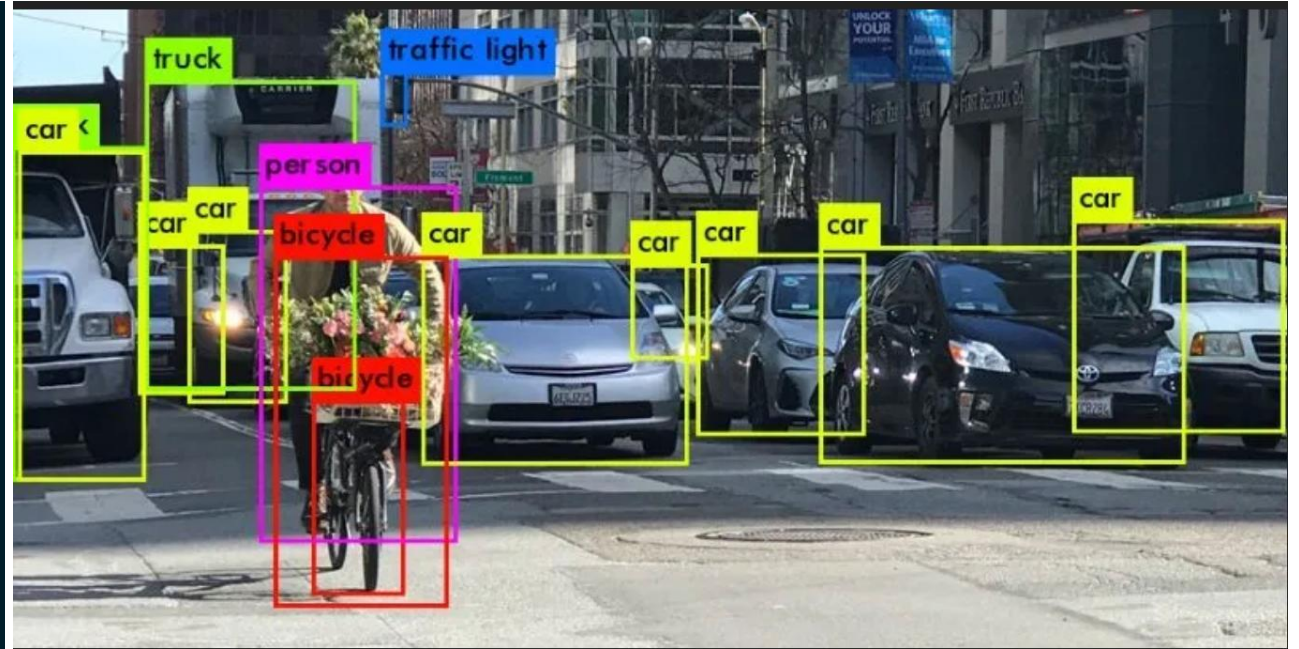
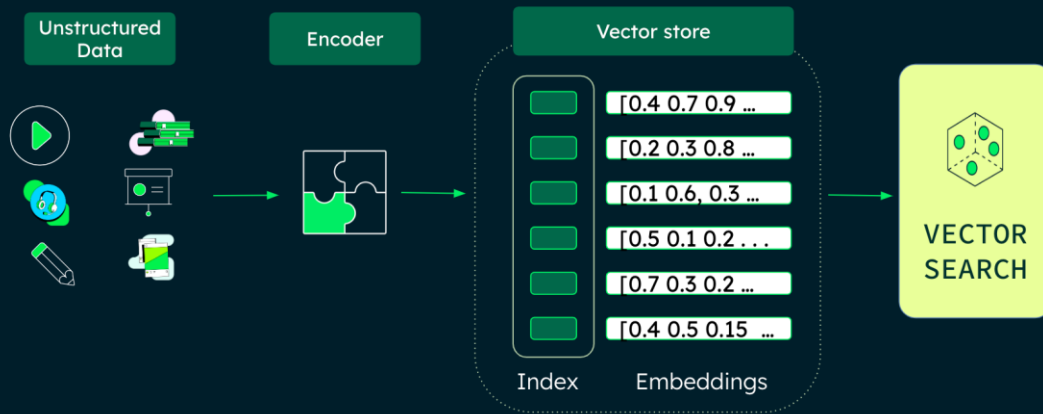
Algoritmos matemáticos:

- Similitud del coseno: se basa en el ángulo entre dos vectores, sin importar su tamaño.
- Distancia euclidiana: distancia en línea recta entre dos vectores en el espacio.
- Similitud del producto de puntos: mide el producto de las magnitudes de los vectores y del coseno entre ellos.

Los datos se convierten a vectores utilizando modelos de codificación.

## Bases de datos vectoriales

Vector Search engines use distances in the embedding space to represent similarity



<https://www.mongodb.com/es/resources/basics/databases/vector-databases>

Bases de datos de documentos optimizadas  
para búsquedas vectoriales

# Evolución BBDD

- 0. Almacenado de registros
- 1. Almacenado secuencial de registros
- 2. BBDD jerárquicas
- 3. BBDD red
- 4. BBDD relacionales

- 5. BBDD orientadas a objetos
- 6. BBDD NoSQL
- 7. BBDD columnar
- 8. BBDD orientadas a grafos

9. BBDD de series temporales, BBDD de libro mayor y BBDD en memoria

10. BBDD autónomas

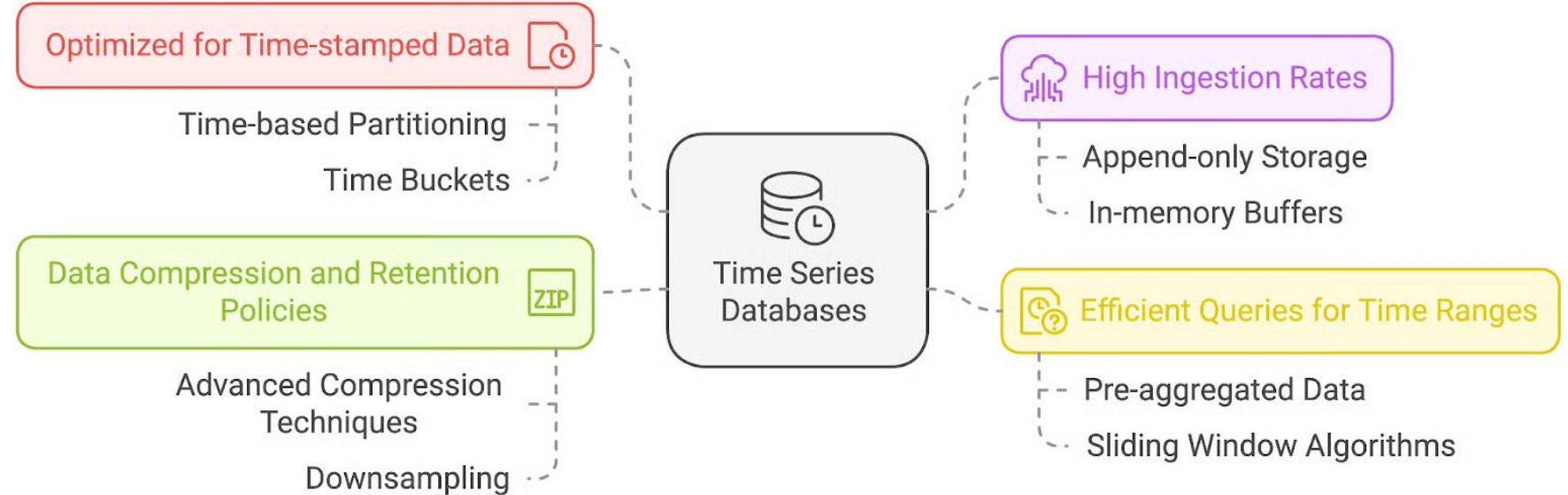


## 9. Bases de datos de series temporales y de libro mayor

Para gran cantidad de datos. Rápidas y escalables.

### Series temporales (TSDB)

Cada punto de datos incluye una marca de tiempo usada como índice.



## 9. Bases de datos de series temporales y de libro mayor

Para gran cantidad de datos. Rápidas y escalables.

**De libro mayor (ledger):** registro de transacciones seguro y verificable, de bajo nivel de acceso.

Se usan para estudios de medicamentos, seguimientos de procesos registrales complejos, cumplimiento legal y fiscal, ledger de criptomonedas.

Ejemplos: Amazon Quantum Ledger Database (Amazon QLDB), IBM LM.

- ✓ Inmutabilidad: impiden el borrado o modificación de datos históricos
- ✓ Verificación: verificable criptográficamente
- ✓ Seguridad: protege la integridad del historial

# Evolución BBDD

Current table Provisioned at: WCU=750 and RCU=300

| Primary Key   |              | Attributes        |            |     |
|---------------|--------------|-------------------|------------|-----|
| Partition Key | Sort Key     | Radiant Intensity | Wavelength | ... |
| 2018-03-15    | 00:00:00.002 | 17.372 W/Sr       | 713 nm     | ... |
| 2018-03-15    | 00:00:00.004 | 17.385 W/Sr       | 712 nm     | ... |
| 2018-03-15    | 00:00:00.005 | 17.478 W/Sr       | 708 nm     | ... |
| 2018-03-15    | 00:00:00.007 | 19.172 W/Sr       | 674 nm     | ... |
| ...           | ...          | ...               | ...        | ... |

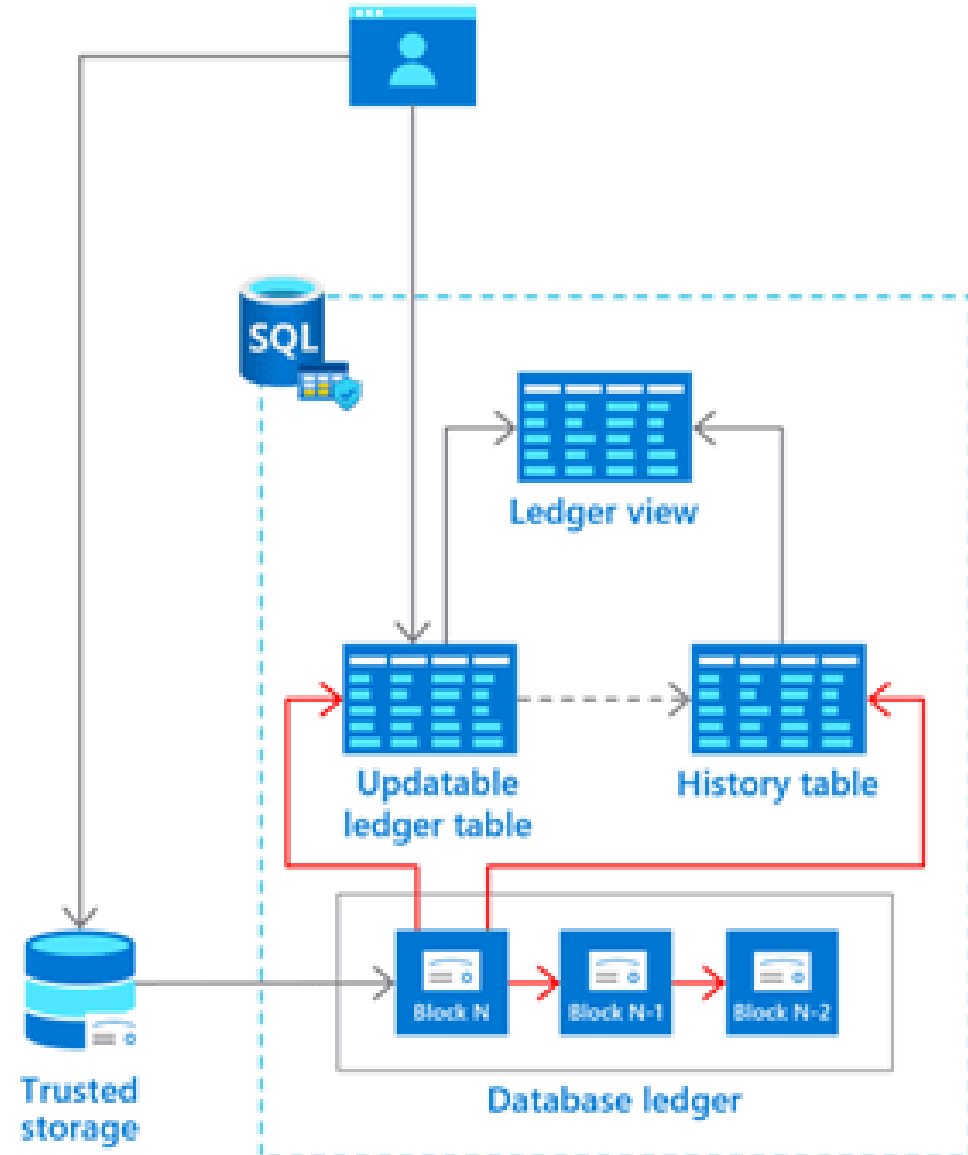
Previous table Provisioned at: WCU=1 and RCU=100

| Primary Key   |              | Attributes        |            |     |
|---------------|--------------|-------------------|------------|-----|
| Partition Key | Sort Key     | Radiant Intensity | Wavelength | ... |
| 2018-03-14    | 00:00:00.001 | 16.473 W/Sr       | 512        | ... |
| 2018-03-14    | 00:00:00.003 | 16.489 W/Sr       | 519        | ... |
| 2018-03-14    | 00:00:00.004 | 16.814 W/Sr       | 522        | ... |
| 2018-03-14    | 00:00:00.006 | 16.719 W/Sr       | 506        | ... |
| ...           | ...          | ...               | ...        | ... |

Older table Provisioned at: WCU=1 and RCU=1

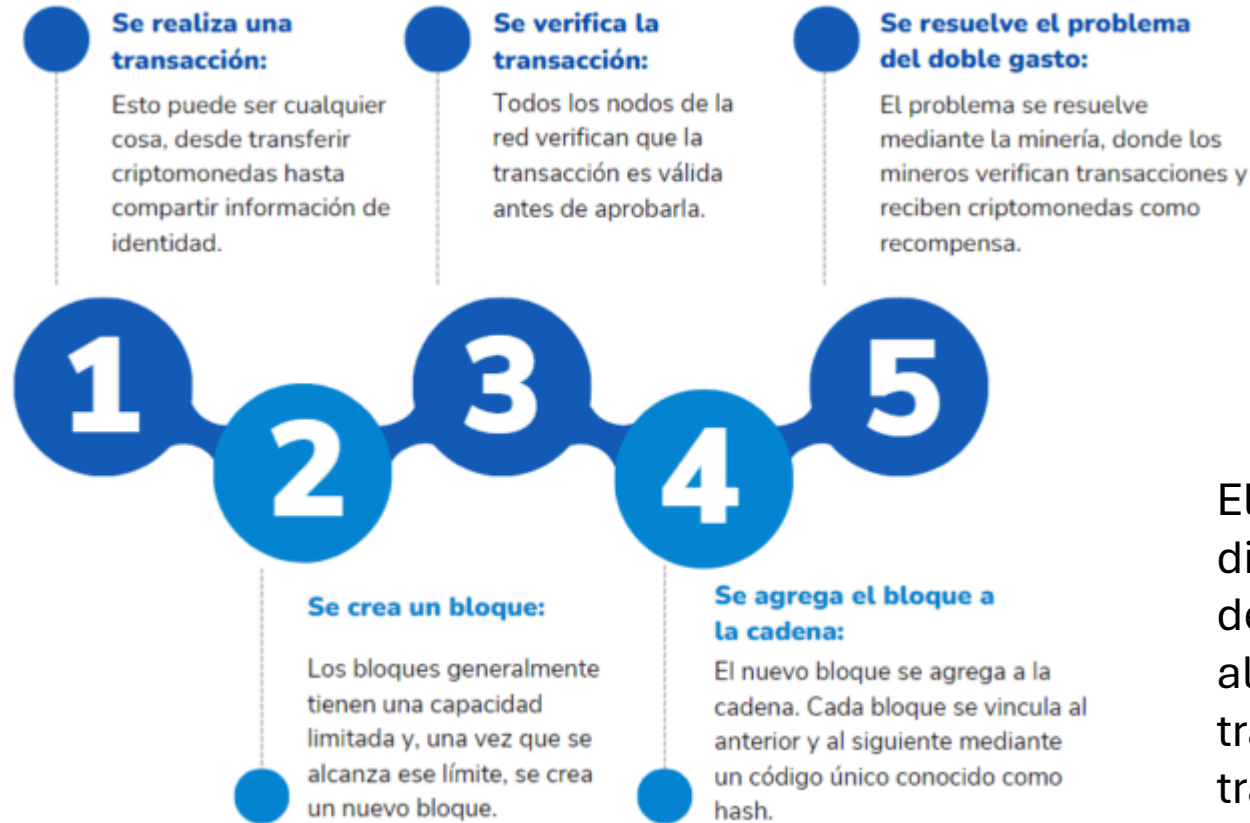
| Primary Key   |              | Attributes        |            |     |
|---------------|--------------|-------------------|------------|-----|
| Partition Key | Sort Key     | Radiant Intensity | Wavelength | ... |
| 2018-03-10    | 00:00:00.001 | 13.669 W/Sr       | 456        | ... |
| 2018-03-10    | 00:00:00.002 | 13.522 W/Sr       | 459        | ... |
| 2018-03-10    | 00:00:00.004 | 13.596 W/Sr       | 457        | ... |
| 2018-03-10    | 00:00:00.005 | 15.721 W/Sr       | 425        | ... |
| ...           | ...          | ...               | ...        | ... |

AWS. Ejemplo de datos de serie temporal. La tabla actual está aprovisionada con una capacidad de lectura/escritura mayor y las tablas anteriores se han reducido, porque el acceso a ellas es infrecuente.



Esquema de Microsoft Ledger

# Evolución BBDD



BlockchainDB  
=  
Ledger distribuido

El blockchain es un registro descentralizado y distribuido que permite la creación de una cadena de bloques interconectados, cada uno almacenando información de manera segura y transparente. A diferencia de las bases de datos tradicionales, donde la información se almacena en un solo lugar, el blockchain descentraliza la información, lo que lo hace resistente a la manipulación y a ataques cibernéticos.

## 9. Bases de datos en memoria

Para cargas de trabajo con altas tasas de operación y baja latencia.

Optimizadas para el almacenamiento transitorio de datos.

Se almacena en memoria RAM (precio elevado), reduciendo la latencia E/S del disco.

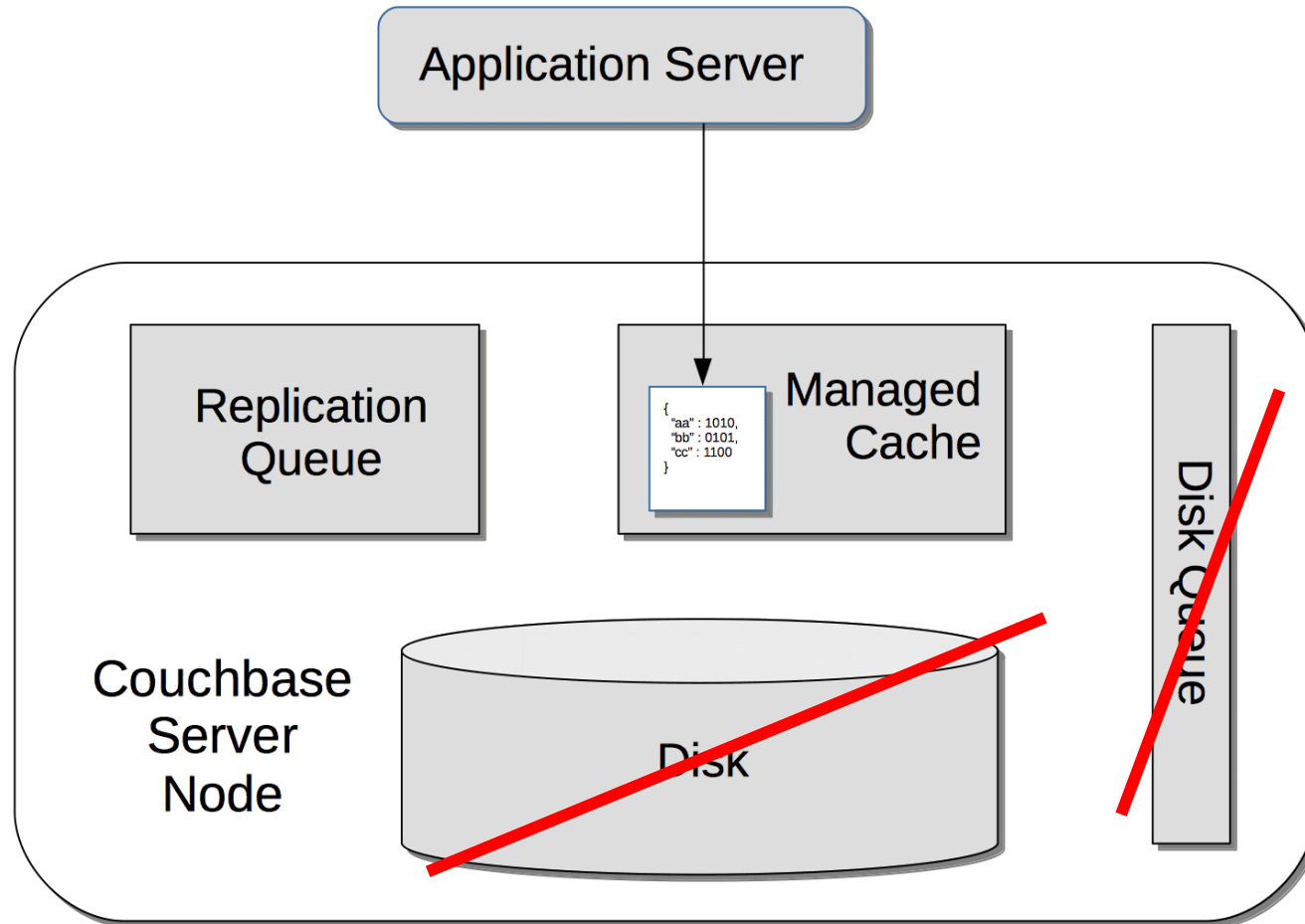
Requieren medidas adicionales de durabilidad: protección para que no se apague el disco y se pierda.

Casos prácticos: análisis en tiempo real, almacenamiento en caché, almacenamiento de sesiones o cualquier cosa transitoria.

Ejemplos: Couchbase, Redis, Memcached, Oracle In-Memory.

Tipo: SQL y NoSQL. A menudo, las empresas utilizan ambos enfoques (persistencia polígloa) para equilibrar rendimiento y consistencia.

## 9. Bases de datos en memoria



Couchbase

- Efímero
- Sólo memoria

# Evolución BBDD

- 0. Almacenado de registros
- 1. Almacenado secuencial de registros
- 2. BBDD jerárquicas
- 3. BBDD red
- 4. BBDD relacionales

- 5. BBDD orientadas a objetos
- 6. BBDD NoSQL
- 7. BBDD columnar
- 8. BBDD orientadas a grafos

9. BBDD de series temporales, BBDD de libro mayor y BBDD en memoria

10. BBDD autónomas

## 10. Bases de datos autónomas o de autogestión

Basadas en la nube.

Utilizan aprendizaje automático e inteligencia artificial para automatizar la gestión de la BD:

- Actualizaciones
- Ajustes
- Copias de seguridad
- Seguridad

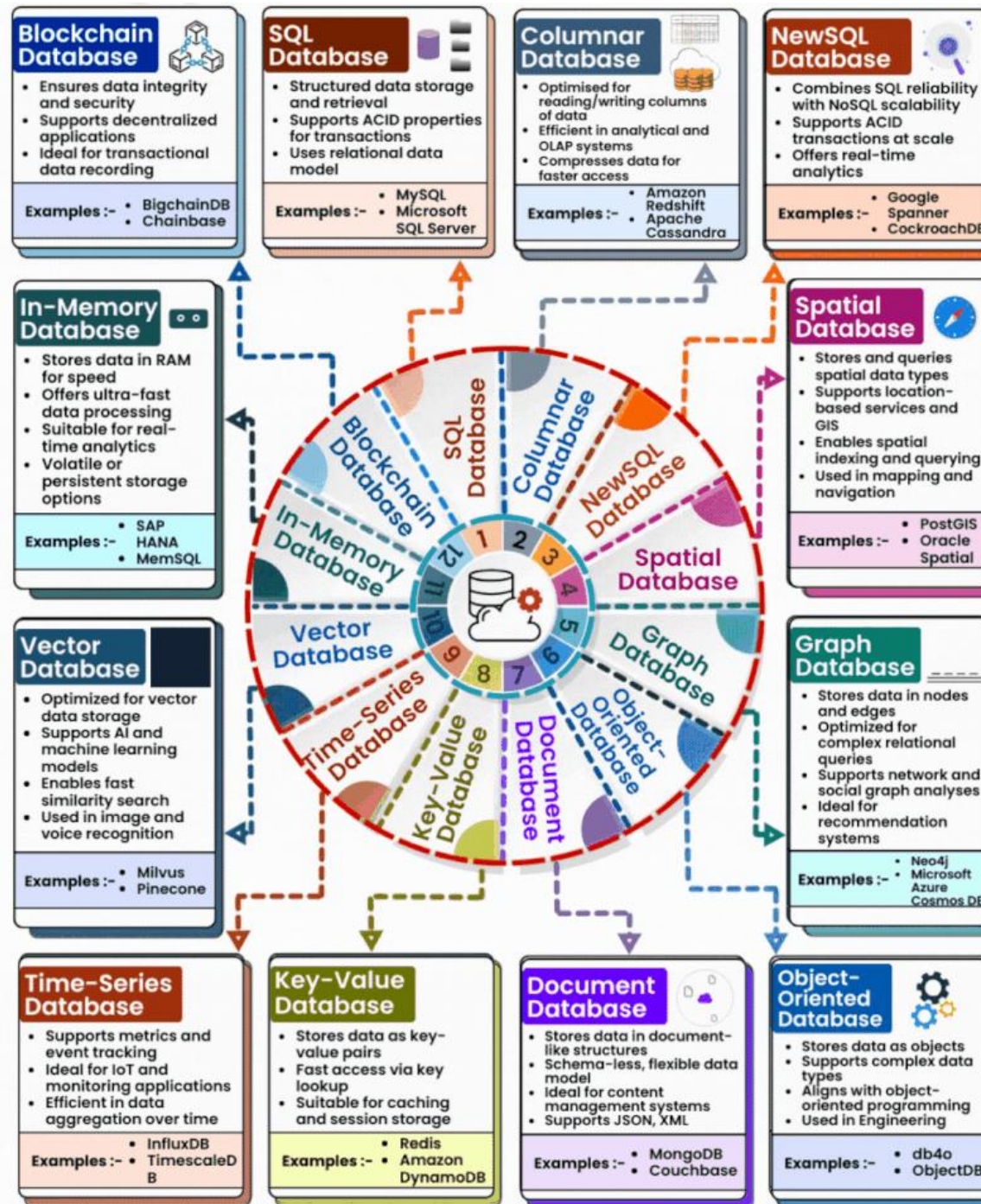
Autogestión + Autoprotección + Autoreparación

Ejemplos

- indexación automática
- escalado automático
- optimización de cargas de trabajo específicas







# ¿SQL o NoSQL?

SQL <-> NoSQL

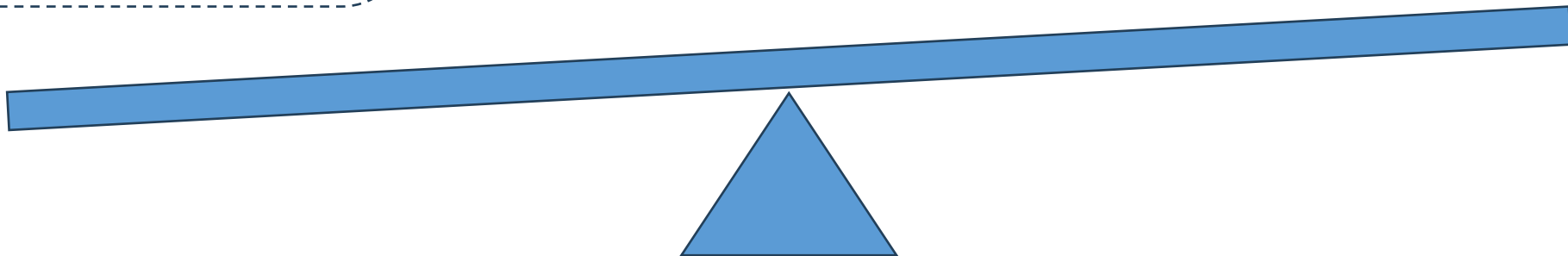
Escalabilidad  
vertical

Consistencia

Necesidades  
específicas de cada  
proyecto o parte de  
proyecto.

Escalabilidad  
horizontal

Flexibilidad



# SQL

| Aspecto                       | Ventajas                                                                    | Desventajas                                                                                  |
|-------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Estructura</b>             | Esquema fijo que organiza y facilita el entendimiento de los datos.         | Cambiar el esquema puede ser complejo y costoso.                                             |
| <b>Consistencia</b>           | Garantiza integridad y confiabilidad mediante propiedades ACID*.            | No es adecuada para aplicaciones donde la disponibilidad prevalece sobre la consistencia.    |
| <b>Consultas</b>              | Soporte para consultas complejas (JOIN, GROUP BY, subconsultas).            | Puede ser ineficiente para grandes volúmenes de datos o consultas en datos no estructurados. |
| <b>Estandarización</b>        | Amplia aceptación del lenguaje SQL, facilitando portabilidad e integración. | Las implementaciones pueden variar entre sistemas (MySQL, PostgreSQL, etc.).                 |
| <b>Madurez</b>                | Décadas de desarrollo con amplia comunidad y documentación.                 | Soluciones comerciales pueden implicar altos costos en licencias y mantenimiento.            |
| <b>Escalabilidad</b>          | Ideal para escalabilidad vertical (aumentar recursos del servidor).         | Escalabilidad horizontal (añadir más servidores) limitada o menos eficiente.                 |
| <b>Datos no estructurados</b> | Buen desempeño en datos estructurados y relaciones claras.                  | Mal rendimiento con datos no estructurados o en constante cambio.                            |

\*ACID  
Atomicidad,  
Consistencia,  
Aislamiento y  
Durabilidad

## NoSQL

| Aspecto                     | Ventajas                                                                                                       | Desventajas                                                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Flexibilidad</b>         | No requiere un esquema fijo, lo que permite manejar datos no estructurados o semi-estructurados.               | La falta de un esquema definido puede dificultar la consistencia y el control de los datos.      |
| <b>Escalabilidad</b>        | Diseñada para escalabilidad horizontal, ideal para manejar grandes volúmenes de datos distribuidos.            | Configurar y mantener la escalabilidad horizontal puede ser complejo.                            |
| <b>Velocidad</b>            | Optimizada para operaciones rápidas en grandes cantidades de datos, especialmente en clave-valor o documentos. | Consultas complejas, como JOIN, no son tan eficientes ni fáciles como en SQL.                    |
| <b>Modelos de datos</b>     | Admite diferentes modelos (clave-valor, documentos, grafos, columnas), adaptándose a diversos casos de uso.    | Elegir el modelo adecuado puede ser confuso para principiantes o proyectos nuevos.               |
| <b>Alta disponibilidad</b>  | Su diseño distribuido garantiza mayor disponibilidad en entornos de red.                                       | Generalmente, sacrifica consistencia en favor de disponibilidad.*                                |
| <b>Costo</b>                | Muchas soluciones NoSQL son de código abierto, reduciendo costos de implementación inicial.                    | Algunos sistemas avanzados requieren inversión significativa en infraestructura y capacitación.  |
| <b>Desempeño específico</b> | Ideal para datos que cambian rápidamente o que no requieren relaciones complejas.                              | Menor rendimiento en aplicaciones que necesitan consistencia fuerte o relaciones bien definidas. |

\*Consistencia eventual



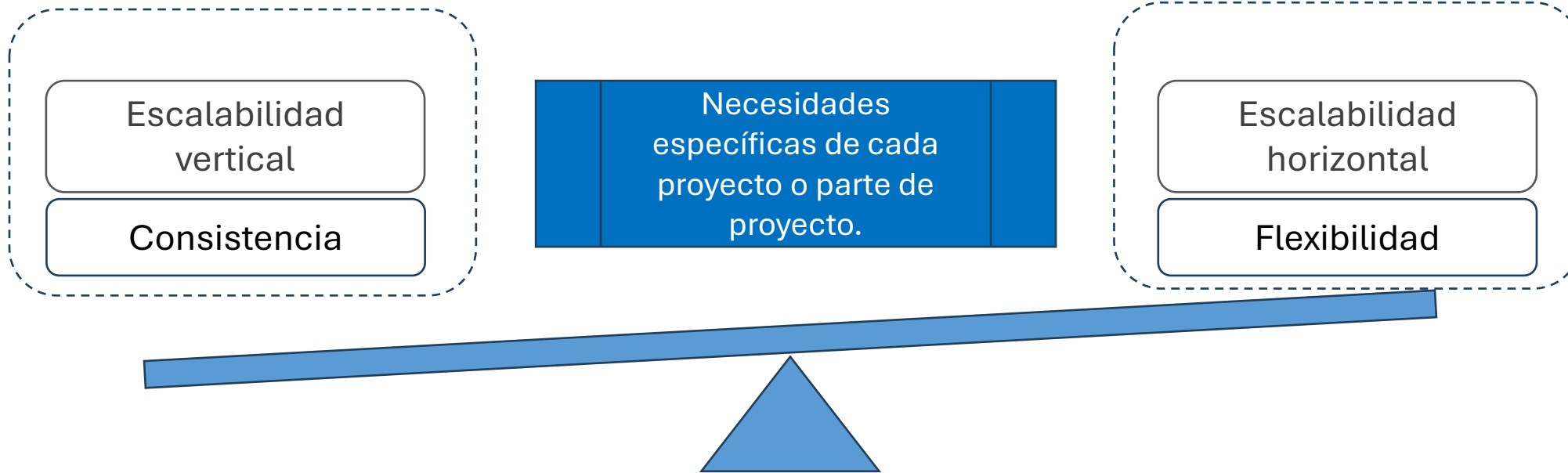
# ¿SQL o NoSQL?



| Aspecto         | SQL                                          | NoSQL                                                      |
|-----------------|----------------------------------------------|------------------------------------------------------------|
| Modelo de Datos | Relacional (tablas con filas y columnas)     | No relacional (clave-valor, documentos, grafos, columnas)  |
| Esquema         | Rígido y predefinido                         | Dinámico y flexible                                        |
| Escalabilidad   | Escalabilidad vertical (mejorando hardware)  | Escalabilidad horizontal (agregando nodos)                 |
| Flexibilidad    | Menor flexibilidad para cambios en los datos | Alta flexibilidad para datos en evolución                  |
| Tipos de Datos  | Datos estructurados                          | Datos no estructurados, semi-estructurados y estructurados |
| Casos de Uso    | ERP, CRM, aplicaciones financieras           | IoT, redes sociales, análisis de Big Data                  |

# ¿SQL o NoSQL?

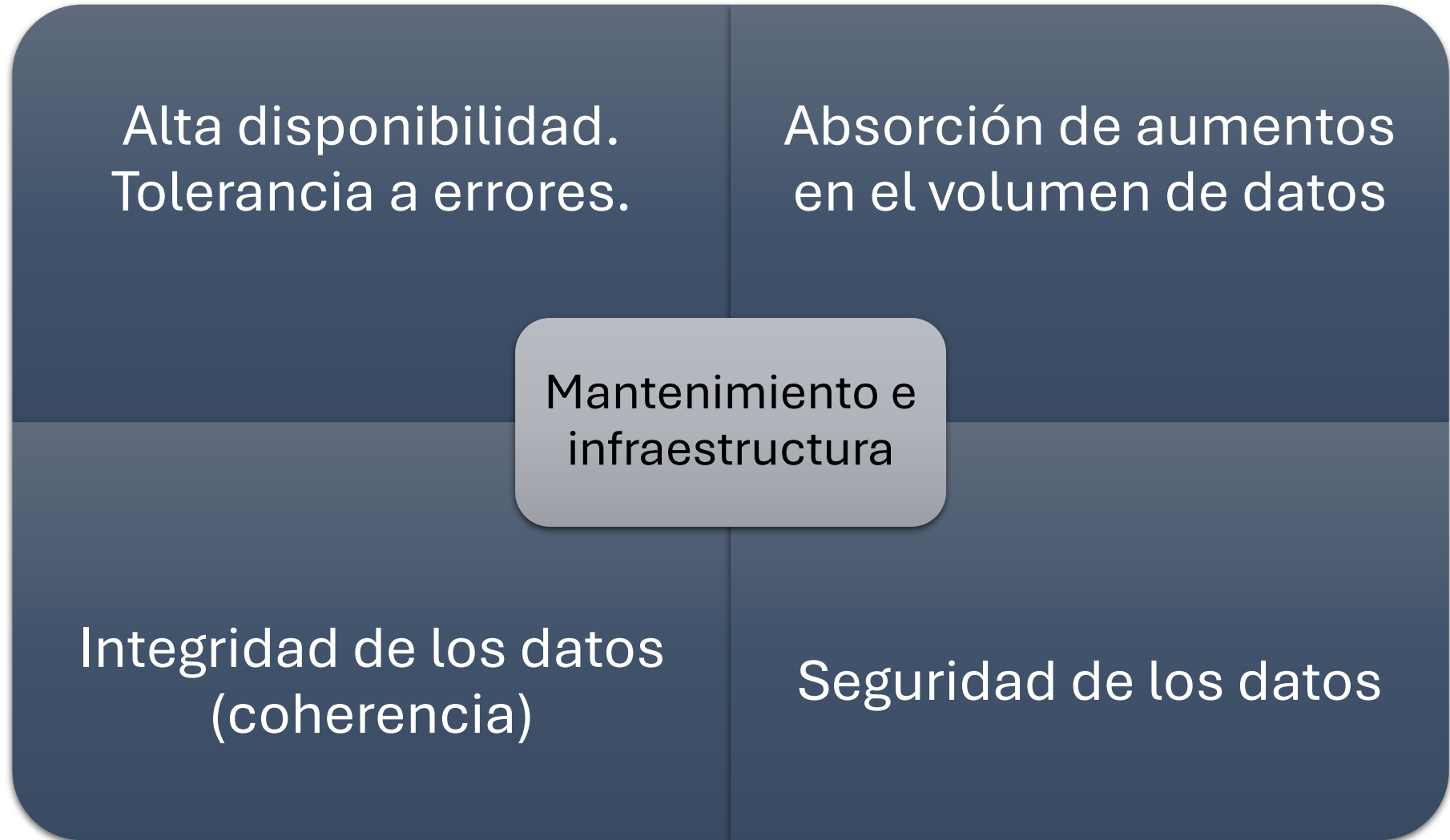
SQL <-> NoSQL



**Futuro:** convergencia de SQL y NoSQL en soluciones híbridas => mejor adaptación de las empresas a las necesidades cambiantes, aprovechando las ventajas de ambos sistemas.

## **Desafíos BBDD**

## Desafíos BBDD





- Consultas complejas -> respuestas muy rápidas => ↓ latencia

Alta disponibilidad de las BD

- accesible y operativa durante largos períodos
- minimizar los tiempos de inactividad o interrupción del servicio

¿Causas?

## Desafíos BBDD

- Consultas complejas -> respuestas muy rápidas => ↓ latencia

Alta disponibilidad de las BD

- accesible y operativa durante largos períodos
- minimizar los tiempos de inactividad o interrupción del servicio

Causas:

- .- mantenimiento
- .- fallos de hardware, errores de software
- .- ataques cibernéticos
- .- desastres naturales / daños infraestructura

¿Estrategias aplicables?

- Consultas complejas -> respuestas muy rápidas => ↓ latencia

Alta disponibilidad de las BD

- accesible y operativa durante largos períodos
- minimizar los tiempos de inactividad o interrupción del servicio

Estrategias aplicables:

- redundancia de hardware
- replicación de datos: síncrona o asíncrona, según los requisitos de consistencia y rendimiento
- partición de datos
- balanceo de carga
- planes de recuperación: copias de seguridad -> ubicación, acceso y procedimientos de restauración
- \* virtualización

- Alta disponibilidad de las BD
- Absorción de aumentos significativos en el volumen de datos:
  - administrar y organizar de forma eficiente
  - respuesta en tiempos cortos a los cambios en la demanda
  - límites en la escalabilidad: vertical y horizontal

## Desafíos BBDD

- Alta disponibilidad de las BD
- Absorción de aumentos significativos en el volumen de datos:
  - administrar y organizar de forma eficiente
  - respuesta en tiempos cortos a los cambios en la demanda
  - límites en la escalabilidad: vertical y horizontal
- consultas e índices eficientes
- almacenamiento en caché de datos de consultas
- gestión de distribución y escalado

## Desafíos BBDD

- Alta disponibilidad de las BD
- Absorción de aumentos significativos en el volumen de datos:
  - administrar y organizar de forma eficiente
  - respuesta en tiempos cortos a los cambios en la demanda
  - límites en la escalabilidad
- Integridad de los datos: coherencia
- Garantía de seguridad de los datos: datos seguros pero fácilmente accesibles para los usuarios.
- Mantenimiento e infraestructura

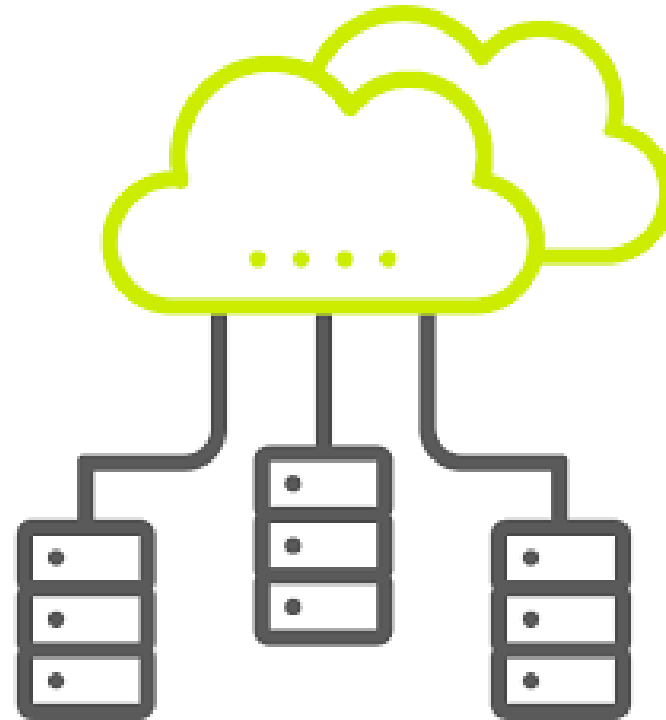
## **BBDD distribuidas**

## BBDD distribuidas

Sistema de gestión de bases de datos en el que las bases de datos no se encuentran centralizadas en un único servidor.

Los servidores (nodos) pueden estar en la misma localización física o en distintas localizaciones distribuidas a través de una red geográficamente.

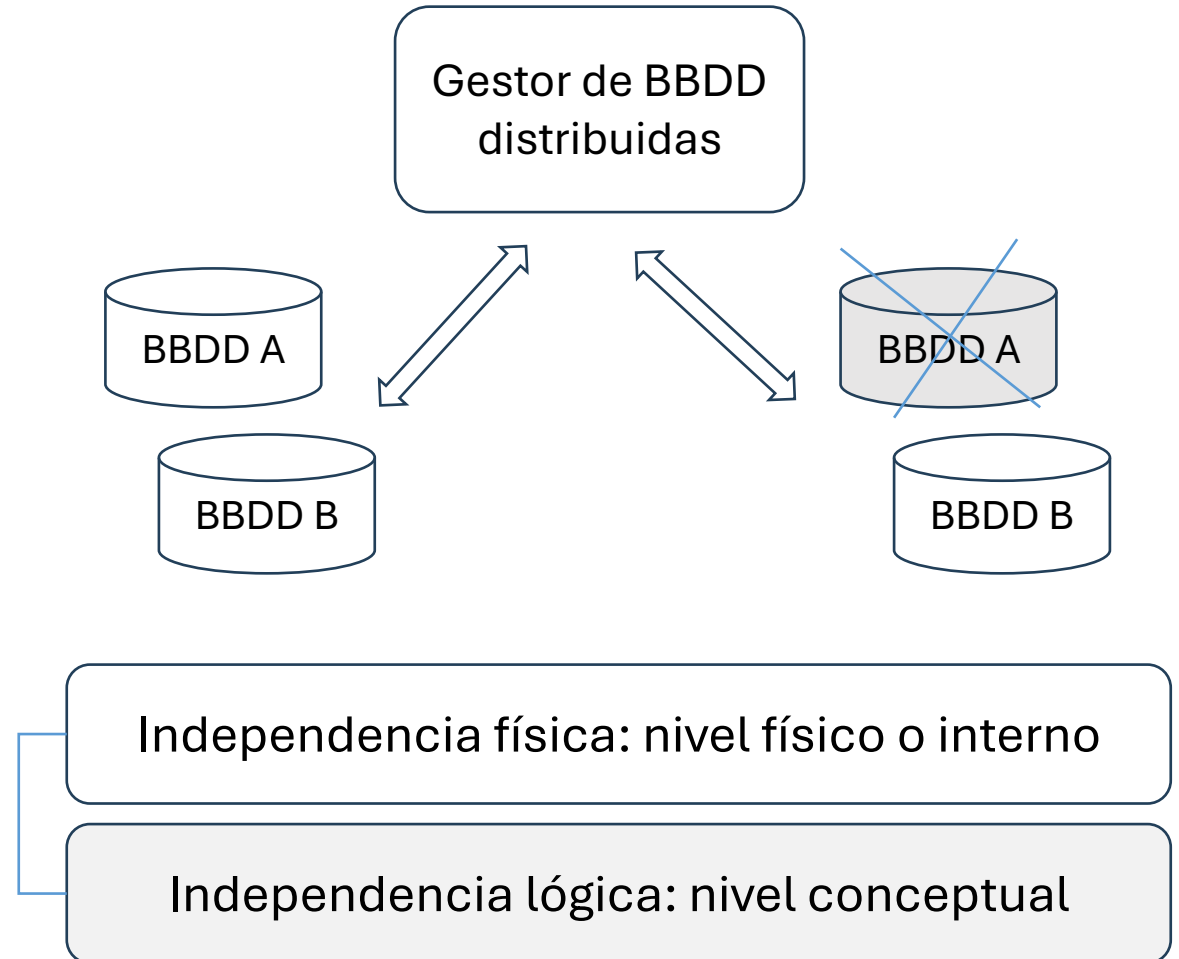
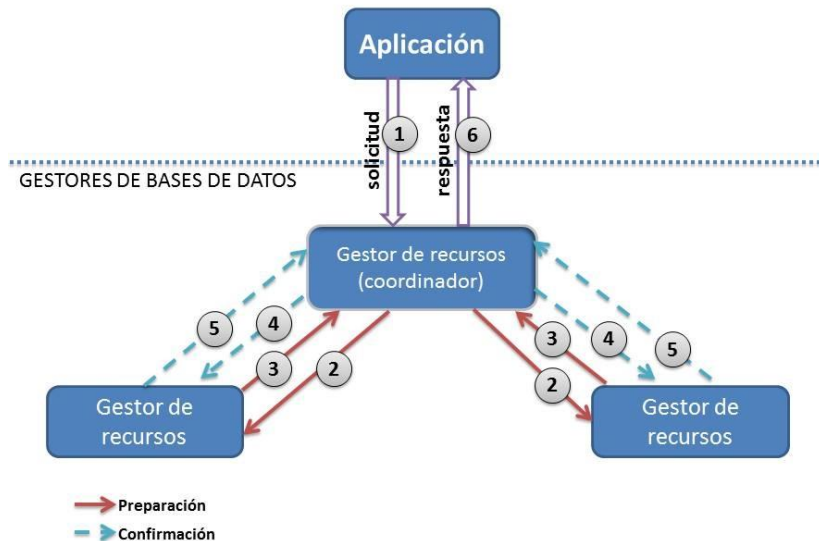
¿Pueden estar en cualquier localización geográfica?





## Ventajas

- Escalabilidad: escalado horizontal
- Tolerancia a fallos: réplicas
- Rendimiento: datos distribuidos



## Desafíos

- Consistencia: mecanismos de sincronización y resolución de conflictos
- Administración: más compleja
  - Monitoreo constante de los nodos
  - Configuración de la replicación

\* Consistencia eventual: la consistencia eventual en NoSQL asegura que, con el tiempo, todas las lecturas de un registro devolverán el último valor confirmado, aunque pueda haber inconsistencias temporales.

Actualización y réplicas: estrategias de replicación de datos

## Actualización y réplicas: estrategias de replicación de datos

- **Replicación síncrona:** cada vez que se realiza una modificación en la base de datos, se espera a que todos los nodos de replicación confirmen la operación antes de considerarla exitosa.
  - Alta consistencia
  - Aumenta la latencia

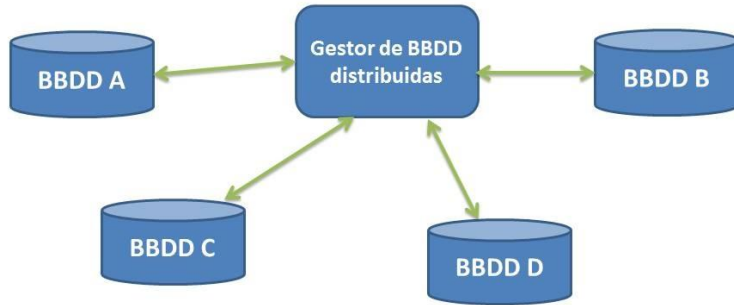
## Actualización y réplicas: estrategias de replicación de datos

- **Replicación síncrona:** cada vez que se realiza una modificación en la base de datos, se espera a que todos los nodos de replicación confirmen la operación antes de considerarla exitosa.
  - Alta consistencia
  - Aumenta la latencia
- **Replicación asíncrona:** los cambios se propagan de forma asíncrona.
  - Inconsistencias temporales en los datos
  - Mejor rendimiento

Actualización y réplicas: estrategias de replicación de datos

- **Replicación síncrona**
- **Replicación en cascada:** los cambios se propagan de un nodo a otro en forma de cascada.
  - Aumenta el riesgo de fallos en la cadena y puede aumentar la latencia.
- **Replicación asíncrona**
- **Replicación por grupos:** en nodos agrupados en grupos lógicos.
  - Muy complejo de administrar, menos usado.

## Actualización y réplicas: estrategias de replicación de datos



Las bases de datos distribuidas por naturaleza (ej. MongoDB) ya consideran estas estrategias en la gestión. Para aplicarlo en BD de tipo SQL es conveniente usar un gestor de BD distribuidas que facilite el proceso

### **Gestor de transacciones de Db2 (IBM)**

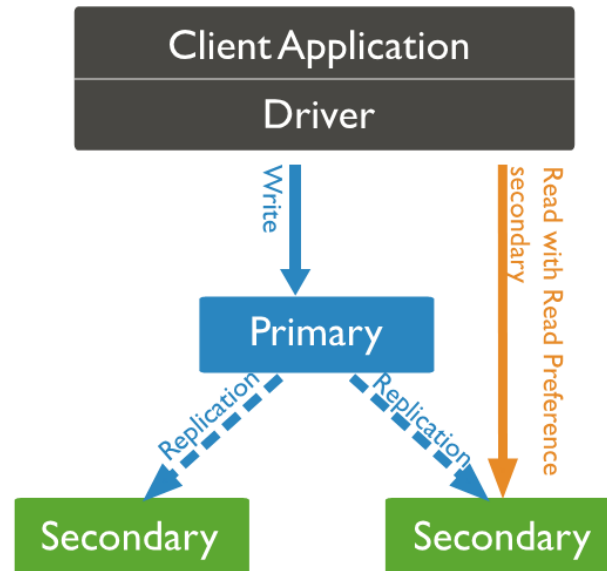
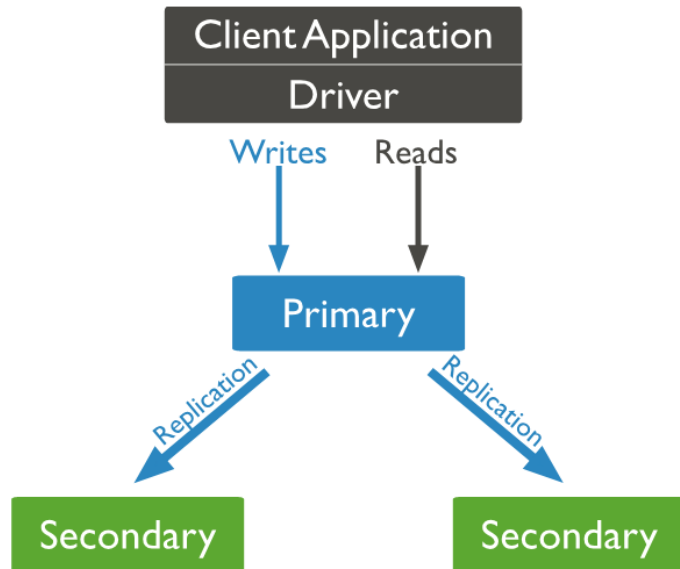
El gestor de transacciones (TM) Db2 asigna identificadores a las transacciones, supervisa su progreso y se responsabiliza de la finalización y anomalía de las transacciones. Db2 TM almacena información de transacción en la base de datos TM designada.

### **Coordinador de transacciones distribuidas (DTC) para Azure SQL Managed Instance (Microsoft)**

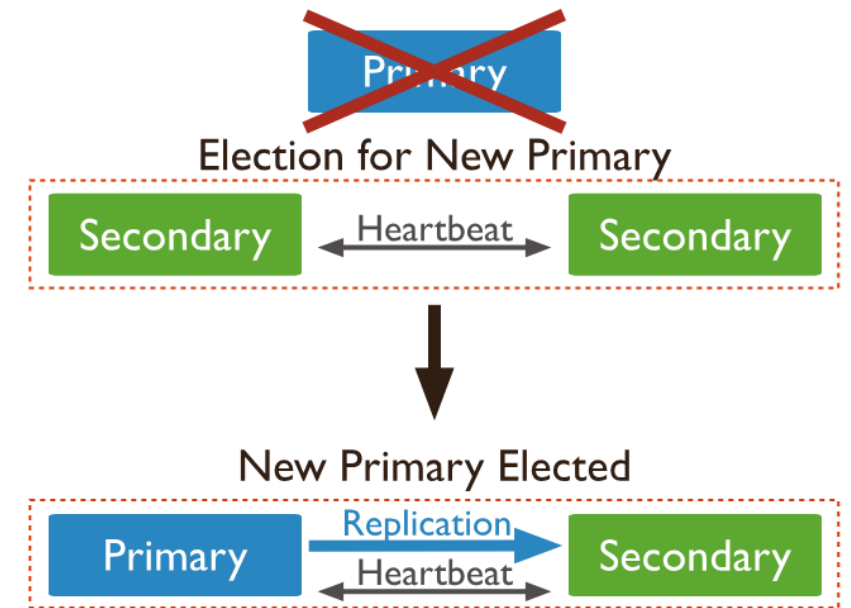
Permite ejecutar transacciones distribuidas en varios entornos que pueden establecer la conectividad de red con Azure. Azure se encarga de la administración y el mantenimiento, como el registro, el almacenamiento, la disponibilidad de DTC y las redes.

# BBDD distribuidas

Actualización y réplicas: estrategias de replicación de datos, ejemplo MongoDB



Opcional





# Comentarios y preguntas